

Structure-Aware Adversarial Image Generation

From Evolutionary Algorithms to Deterministic Pixel
Selection

Tidiane Ciavarella

Student ID: 019152101A

University of Luxembourg

Programme: Applied Information Technology

Study Director: Mr. Volker MÜLLER



Submission Date: June 7, 2025

Number of Pages: 81

Number of Words: 16,491

Declaration of Authorship

I hereby declare that I am the sole author of the work entitled:

**Structure-Aware Adversarial Image Generation:
From Evolutionary Algorithms to Deterministic Pixel Selection**

and here enclosed, and that I have compiled it in my own words. I confirm that I have not used any other than the cited sources and aids, and that all parts of this work, which I have adopted from other sources, are acknowledged and designated as such. I also confirm that this work has not been submitted previously or elsewhere.

Date: June 7, 2025

First name and surname: Tidiane Ciavarella

Signature:



Abstract

Adversarial images intentionally crafted to mislead convolutional neural networks (CNNs) represent a critical vulnerability in modern deep learning systems. In this work, we explore and compare several complementary strategies for generating such adversarial examples.

We begin by introducing a structural, edge-guided evolutionary algorithm that restricts perturbations to the object contours within an image. This method provides a perceptually aware mechanism for adversarial noise generation, leveraging edge maps to constrain the mutation space.

Next, we extend this framework to create *universal adversarial masks*, which are image-agnostic perturbations capable of inducing targeted misclassifications across a wide range of inputs. We evaluate their effectiveness and discuss the challenges involved in optimizing a fixed perturbation under strict visual constraints.

Building on this, we propose a color-space-aware variant of the evolutionary algorithm that operates in the YCbCr domain. By applying noise to luminance and chrominance channels separately, then converting and clamping back to RGB, we achieve fine control over visual distortion while maintaining adversarial effectiveness.

Finally, we develop a deterministic algorithm called *best_pixels*, which mimics gradient-like behavior through iterative selection and modification of the most influential pixels. This method offers both interpretability and reproducibility, and we demonstrate its versatility across multiple target labels and input samples.

All experiments were conducted using the same ResNet-50 architecture and a consistent hardware configuration to ensure reproducibility and fair comparison. Together, these methods provide a multi-faceted analysis of adversarial robustness, offering insights into both the potential and the limitations of various attack strategies.

Contents

1	From Pixels to Intelligence: A Gentle Introduction	6
1.1	What Is an Image?	6
1.2	From Color to RGB: How Screens Display Visual Data	7
1.3	Representing Images as Matrices	8
1.4	RGB Images and 3D Matrices	10
1.5	From Images to Predictions: Introduction to CNNs	12
1.6	Adversarial Images: When Machines Are Fooled	14
2	Edge-Guided Perturbations for Adversarial Image Generation	17
2.1	Submatrix Selection and Edge Detection	17
2.1.1	Motivation and Conceptual Intuition	17
2.1.2	Edge Detection Using Sobel Convolution	18
2.1.3	Perturbation Generation and Smoothing	21
2.2	Evolutionary Algorithms	22
2.2.1	Initialization and Parameterization	22
2.2.2	Evaluation and Selection Strategy	24
2.2.3	Fitness Scoring with CNN Predictions	26
2.2.4	Crossover and Iterative Evolution	27
2.3	Experimental Objective and Setup	30
2.3.1	Experimental Configuration	30
2.3.2	Objective of the Experiment	32
2.3.3	Execution Time Benchmarking	33
2.3.4	First Experience	33
2.3.5	Second Experience	35
2.3.6	Conclusion	37
3	Evolutionary Generation of Universal Adversarial Masks	39
3.1	Universal Adversarial Mask Generation	39
3.1.1	Motivation and Noise Construction	39
3.1.2	Mask Application via Alpha Blending	41
3.2	Experimental Objective and Setup	42
3.2.1	Objective of the Experiment	42

3.2.2	Execution Time Benchmarking	43
3.2.3	First Experience	43
3.2.4	Second Experience	45
3.2.5	Conclusion	47
4	Adversarial Image Generation in the YCbCr Color Space	49
4.1	Color Space Transformation and Algorithm Integration	49
4.1.1	From RGB to YCbCr and Back	50
4.1.2	Algorithm Adaptation	50
4.2	Experimental Objective and Setup	52
4.2.1	Objective of the Experiment	52
4.2.2	Execution Time Benchmarking	53
4.2.3	First Experience	53
4.2.4	Second Experience	54
4.2.5	Third Experience	57
4.2.6	Conclusion	61
5	Deterministic Pixel-Wise Adversarial Image Generation	62
5.1	Exhaustive Binary Pixel Probing Strategy	62
5.1.1	Introduction	63
5.1.2	Algorithm Description	63
5.1.3	Advantages, Constraints, and Precision	63
5.2	Experimental Objective and Setup	64
5.2.1	Objective of the Experiment	65
5.2.2	Execution Time Benchmarking	65
5.2.3	First Experience	66
5.2.4	Second Experience	67
5.2.5	Third Experience	68
5.2.6	Fourth Experience	70
5.2.7	Fifth Experience	73
5.2.8	Conclusion	74
5.3	General Conclusion	75

List of Figures

1.1	An image composed of 529 pixels arranged in a 23×23 grid.	7
-----	--	---

1.2	Close-up of a television pixel revealing the individual red, green, and blue subpixels (LEDs). Source : [7].	8
1.3	Example of a 2D matrix representation with dimensions 8×8	9
1.4	Grayscale image of size 12×16 (left), followed by its pixel intensity values (center), and corresponding 2D matrix representation (right). Source : [9].	10
1.5	3D visualization of a volumetric matrix with dimensions $3 \times 3 \times 3$. Source : [10].	11
1.6	Diagram illustrating a color and its decomposition into red, green, and blue channel intensities.	12
1.7	Simplified schematic of a forward pass through a Convolutional Neural Network.	13
1.8	Comparison of class probabilities from a CNN for a clean image versus its adversarial counterpart.	14
1.9	Visualization of an adversarial image alongside a version with exaggerated pixel mutations for perceptual clarity.	15
2.1	Image representation split into four submatrices obtained using a divider value of 2.	18
2.2	Visualization of the horizontal Sobel kernel used for edge detection. . . .	19
2.3	Vertical Sobel kernel (left) followed by two diagonal Sobel kernels (right) for gradient-based edge extraction.	19
2.4	Decomposition of an image into 12 sub-matrices, four per channel, using a divider value of 2.	21
2.5	Visual impact of the reach parameter controlling the saturation of injected noise in adversarial images.	23
2.6	Effect of increasing the divider parameter on the precision of noise localization, with a constant noise ratio of 25%.	24
2.7	Illustration of the subdivision into elite, middle, and low selection groups based on edge scores.	26
2.8	Grouping of images based on CNN predictions, highlighting internal divisions by output class likelihood.	27
2.9	Crossover operation between two mutated images, restricted to a 25% region of the image area.	29
2.10	Constant hyperparameters used across all experiments to ensure stability and comparability.	31
2.11	Valeurs testées pour les paramètres divider et min_percentage lors de la première expérience.	34
3.1	Adversarial noise pattern classified as a great white shark (class 285) with 92% confidence.	41

3.2	Range of reach values explored during the initial experiment to assess noise saturation effects.	44
3.3	Overview of fixed and stable hyperparameters used throughout the experiments.	44
4.1	Transformation matrices for RGB to YCbCr and YCbCr to RGB color space conversions.	51
4.2	Adversarial image generated by modifying both chrominance and luminance components.	60
4.3	Adversarial image generated by modifying only the chrominance components (Cb and Cr), with luminance preserved.	60
5.1	List of tested values for the percentage and reach parameters during the initial experiment.	66
5.2	Heatmap of ResNet-50 predictions for class 285 across combinations of percentage and reach parameters.	66

List of Tables

2.1	Best predictions and labels before modification for each image.	32
2.2	Best-performing combinations of divider and min_pourcentage per image, achieving optimal convergence toward the target label within the first 100 epochs.	35
2.3	Comparison of prediction confidence for the target label over 1000 epochs between the best-performing targeted and untargeted strategies per image.	36
2.4	Target label prediction comparison over 1000 epochs between targeted and untargeted algorithms, under a fixed pixel modification ratio.	37
3.1	Top-performing masks for class 285, identified across various reach values over 10,000 epochs.	45
3.2	Predicted confidence scores for class 285 across different images after applying universal adversarial masks optimized for that label.	46
3.3	Comparison of top-1 predicted labels before and after applying a 90%-confidence adversarial mask for class 285.	47
4.1	Progression of target label predictions during the first 100 epochs: comparison between the baseline algorithm and the YCbCr-based variant.	54

4.2	Prediction trajectories for the target label over 1000 epochs: baseline algorithm versus YCbCr-based algorithm.	55
4.3	Prediction trajectories for the target label over 100 epochs: baseline algorithm versus chrominance algorithm.	59
5.1	Predicted confidence for class 285 when modifying the top 90% most influential pixels among 100% of the image, using a step size of 0.01.	68
5.2	Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 50% of the image, with a step size of 0.01.	69
5.3	Predicted confidence for class 285 when modifying the top 90% most influential pixels among a fixed 50% subset of the image, with a step size of 0.01.	70
5.4	Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 25% of the image, using a step size of 0.01.	71
5.5	Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 25% of the image, using a step size of 0.005.	72
5.6	Average number of epochs required to generate adversarial examples reaching over 90% confidence for various target labels, by modifying the top 90% most influential pixels within a randomly selected 50% subset of the image at each iteration, using a step size of 0.01.	73

Chapter 1

From Pixels to Intelligence: A Gentle Introduction

This opening chapter serves as a foundational guide, meant to equip the reader with the essential concepts needed to fully grasp the material presented in the later parts of this work. Rather than assuming any prior knowledge, we will gently walk through the basics of how images are constructed, represented, and interpreted, both by humans and by machines.

1.1 What Is an Image?

This first chapter aims to introduce the reader to the fundamental concepts used throughout this study, to facilitate a better understanding of the material that follows. We shall begin at the very beginning by addressing a seemingly simple but essential question: *What is an image?*

An image can be understood as a collection of *pixels*. When we refer to an image as being of size 100×100 , this means that it is composed of 100 columns, each containing 100 pixels. In total, the image consists of $100 \times 100 = 10,000$ individual pixels. Each pixel represents a tiny unit of visual information, and together these pixels form the complete visual representation that we perceive as an image.

To help you better visualize this concept, consider the following example: an image measuring 23×23 pixels. This means that the image shown below, with dimensions of 23×23 , is composed of $23 \times 23 = 529$ individual pixels (see Figure [1.1](#)).

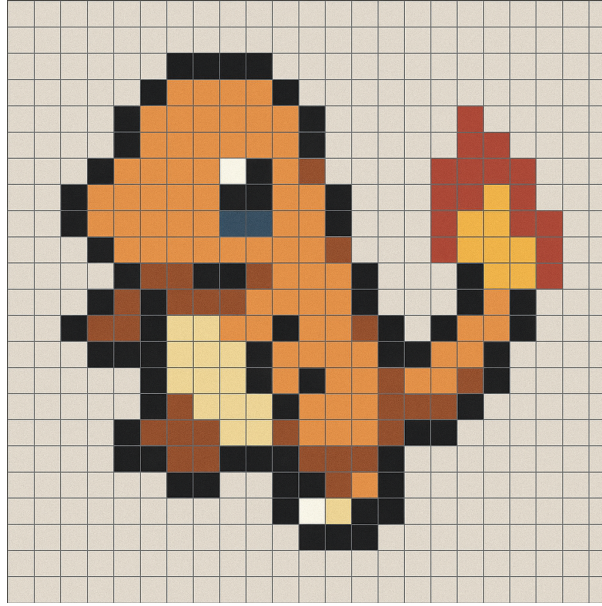


Figure 1.1: An image composed of 529 pixels arranged in a 23×23 grid.

1.2 From Color to RGB: How Screens Display Visual Data

Let us now turn our attention to the *color* of these pixels, as the underlying mechanism behind this aspect may initially appear counterintuitive. As you may already know, any color within the visible spectrum, the colors of the rainbow, can be represented by a combination of the three *primary colors*: red, green, and blue.

This principle is widely used in technology, particularly in the design of digital screens. The reason for this lies in the practicality and efficiency it offers: By relying solely on red, green, and blue **light-emitting diodes (LEDs)**, manufacturers can simplify the design and reduce the production cost of displays. Each pixel on a typical screen is composed of three *subpixels* one red, one green, and one blue (see Figure 1.2). By adjusting the intensity of each of these subpixels, it becomes possible to generate virtually any perceivable color.

Throughout the remainder of this study, we will refer to these three fundamental color components using the acronym **RGB**, which stands for *Red, Green, and Blue*.

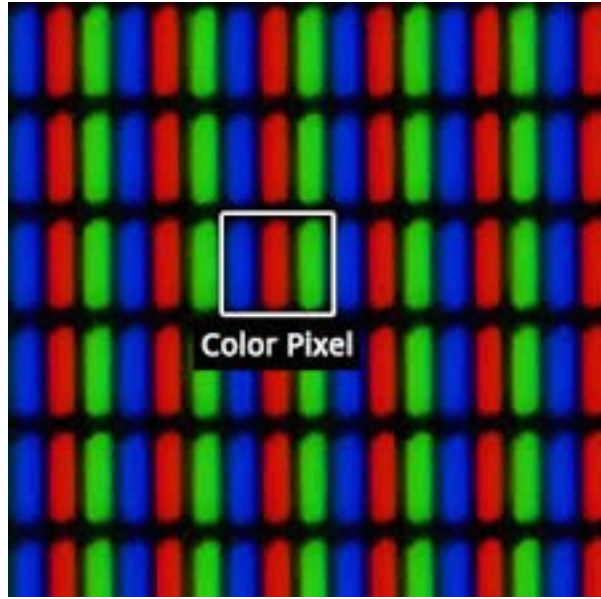


Figure 1.2: Close-up of a television pixel revealing the individual red, green, and blue subpixels (LEDs). Source : [7].

As a result, the **RGB representation** of a pixel at the micro level and of the entire image at the macro level has become the *standard* in the field of digital imaging and display technology.

At this point, you now have a clear idea of what constitutes an image when viewed through a screen: it is made up of pixels, each colored through a specific combination of red, green, and blue light emitted by tiny LEDs.

1.3 Representing Images as Matrices

But this leads to an important question: How do we work with images in computer science? How can something as inherently visual as an image be represented and manipulated using something as abstract and non-visual as lines of code?

The answer lies in the use of *matrices*.

Although matrices might initially seem like a complex mathematical concept, they are, in fact, quite simple to grasp. A two-dimensional matrix is merely a grid, a two-dimensional array filled with numerical values (see Figure 1.3).

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

Figure 1.3: Example of a 2D matrix representation with dimensions 8×8 .

This should remind you of the image presented earlier. However, instead of containing colors, in a matrix, each cell holds a numerical value. Thus, a 8×8 matrix contains 64 numbers, just as an 8×8 image consists of 64 pixels.

This brings us to an important question: How can colors be represented using numbers?

The standard convention is to represent the lowest possible intensity with the value 0 and the highest possible intensity with the value 255. This creates a scale of 256 levels of intensity for each pixel.

As a result, a *black and white image* (also known as a *grayscale image*) can be represented by a matrix, where each value in the matrix falls within the range of 0 to 255 (see Figure 1.4).

A value of 0 corresponds to a completely black pixel. A value of 255 corresponds to a completely white pixel. Any value between these two extremes corresponds to a varying shade of gray.

This simple yet effective encoding allows computers to represent, store, and process images using numerical data.

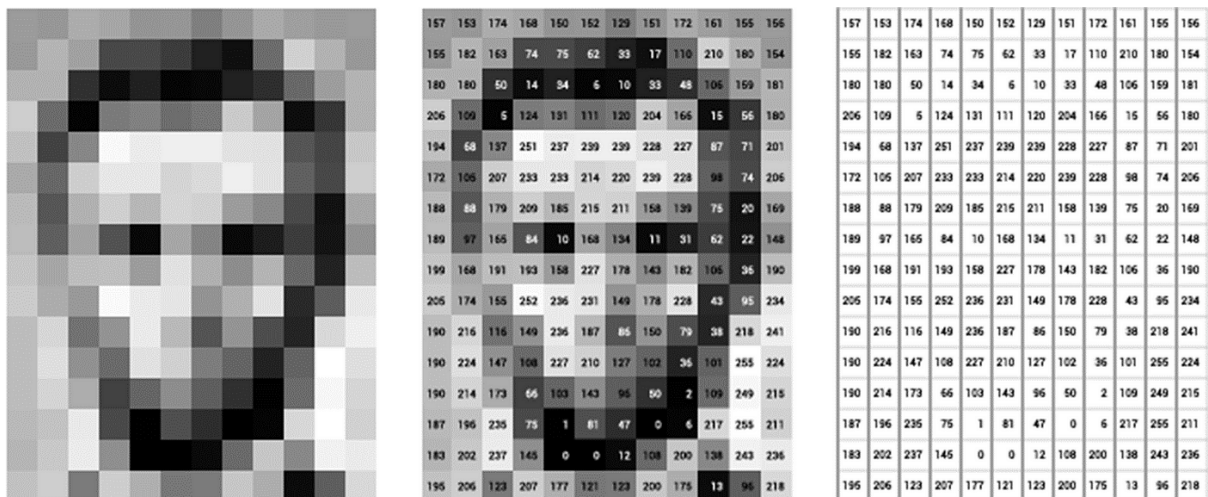


Figure 1.4: Grayscale image of size 12×16 (left), followed by its pixel intensity values (center), and corresponding 2D matrix representation (right). Source : [9].

1.4 RGB Images and 3D Matrices

In order to convert simple numerical values into the light intensities of the LEDs on your screen, we make use of a Python library known as **PIL**, which stands for *Python Imaging Library*. This library handles the conversion of matrices into actual images and displays them directly on your screen.

By now, the general concept of digital images should be clearer. However, an essential question remains unanswered. So far, all the images we have represented using matrices have been black and white. But how can we represent a color image?

One might intuitively suggest using three separate matrices: One for red, one for green, and one for blue. This approach is valid; each matrix would then represent the intensity of one of the three primary RGB color channels across all pixels.

However, in practice, working with three separate 2D matrices can become inefficient and cumbersome. To address this, we instead use a single *three-dimensional matrix*.

Up to this point, all the matrices we have seen were two-dimensional, with one axis for rows and another for columns. Now, by introducing a third dimension, we can include a depth of three layers, one for each of the red, green, and blue channels (see Figure 1.5).

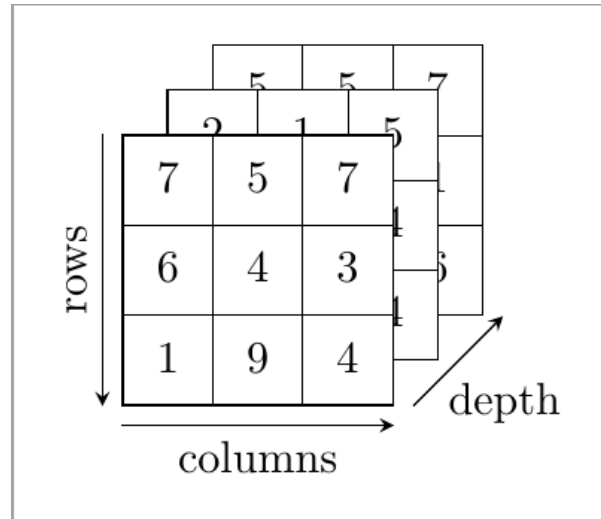


Figure 1.5: 3D visualization of a volumetric matrix with dimensions $3 \times 3 \times 3$. Source : [10].

As you can observe, this matrix has dimensions of $3 \times 3 \times 3$, which means it has a depth of 3, a height of 3, and a width of 3. But where does all this lead us?

As explained above, each pixel on your screen is represented by three individual LEDs, one red, one green, and one blue. To represent an image digitally, we associate the intensity of each of these LEDs with the corresponding values stored in a 3D matrix.

Specifically, the red intensity values are stored in the first 2D slice (depth index 0) of the 3D matrix, the green values in the second slice (depth index 1), and the blue values in the third slice (depth index 2).

For instance, if the 3D matrix shown above represents a 3×3 image composed of 9. The pixel located at the top-right corner of the image would have a red intensity of 7, a green intensity of 5, and a blue intensity of 7.

These three intensity values ($R = 7, G = 5, B = 7$) are interpreted by your screen, which adjusts the brightness of the corresponding RGB LEDs at that pixel's position to reproduce the desired color.

This process is applied across the entire matrix, pixel by pixel, enabling a complete and accurate rendering of a full-color image on your display.

For example, consider the color (100, 56, 203) (see Figure 1.6).

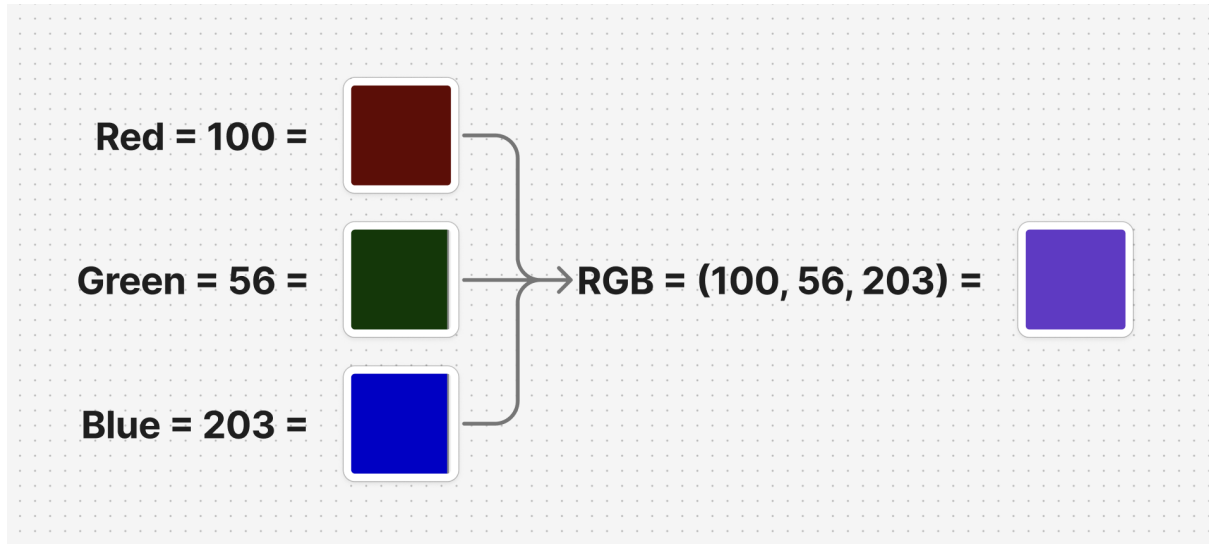


Figure 1.6: Diagram illustrating a color and its decomposition into red, green, and blue channel intensities.

1.5 From Images to Predictions: Introduction to CNNs

Now that you have developed a solid understanding of how images are represented, both in terms of pixels and numerical matrices, we can shift our attention to the second major theme of this study: **artificial intelligence**.

In order to keep this reading experience enjoyable and approachable, I will refrain from delving into overly abstract or complex concepts related to artificial intelligence. Instead, we will focus only on the essential ideas that you need to grasp to fully understand the study at hand.

Let us begin by briefly defining what artificial intelligence actually is. According to widely accepted definitions, artificial intelligence (AI) encompasses a set of techniques that enable machines to simulate certain aspects of human intelligence, such as reasoning, learning, and decision-making. These systems are not intelligent in the human sense, but they are capable of producing outputs that resemble intelligent behavior when provided with input data.

Within this broader framework, we introduce the notion of a *model*. In the context of AI, a **model** refers to a computational system designed to make predictions or decisions based on input data. These predictions can range from forecasting real estate prices to diagnosing medical conditions, depending on the type and complexity of the model.

The specific type of model that concerns us in this study is the **Convolutional Neural Network** (CNN). CNNs are a class of deep learning models particularly well-

suiting for image recognition tasks. Conceptually, they function as pattern extractors: when shown an image, they attempt to determine what it depicts by analyzing the spatial relationships between pixels across multiple layers of processing.

To illustrate this, consider a CNN trained to distinguish between cats and dogs. If we input an image of a dog, the model will output a set of probabilities, perhaps 85% for “dog” and 15% for “cat.” This output represents the model’s confidence in its prediction, and the highest-scoring label is selected as the most likely interpretation of the image.

It is crucial to understand that the model does not “know” what a dog is in the human sense. Instead, it learns to associate certain patterns of pixels with the label “dog” through repeated exposure to labeled examples during training.

In practice, a CNN does not return a definitive *label* (e.g., “dog” or “cat”) but rather a set of *probabilities*. If the model was trained to distinguish between cats and dogs, it might return something like:

- 85% probability that the image shows a dog
- 15% probability that it shows a cat

The sum of all predicted probabilities will always equal 1, reflecting the model’s overall *confidence distribution* (see Figure 1.7).

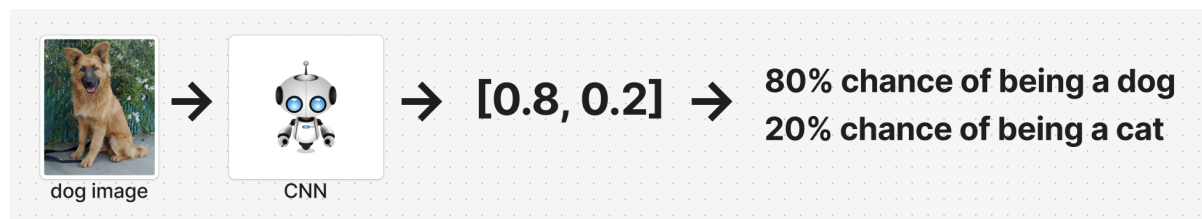


Figure 1.7: Simplified schematic of a forward pass through a Convolutional Neural Network.

In the previous example, the model was limited to two possible options, also called *labels*: dog and cat. However, it is important to note that some models are trained to recognize hundreds or even thousands of labels, including various objects, animals, and even abstract patterns. The number of labels a model can predict depends entirely on the diversity of its training data.

1.6 Adversarial Images: When Machines Are Fooled

At this point, you now have the foundational knowledge necessary to understand what an **adversarial image** is.

An adversarial image is a special type of image that appears perfectly normal and recognizable to a human being, but is specifically designed to deceive a **CNN**. These models sometimes form unexpected and illogical associations between specific pixel patterns, color intensities, and labels.

This leads to a curious phenomenon: By subtly altering just a small portion of an image, for example, an image of a dog, the changes might be so minor that the human eye barely notices them. However, for CNN, these tiny alterations can be drastic, causing it to completely misclassify the image. For example, the model might now predict that the modified image of a dog represents a cat, despite the image still clearly looking like a dog to any human observer (see Figure 1.8).

This type of manipulation reveals both the *vulnerabilities* and the *limitations* of neural networks, and it is precisely this phenomenon that our study seeks to explore.

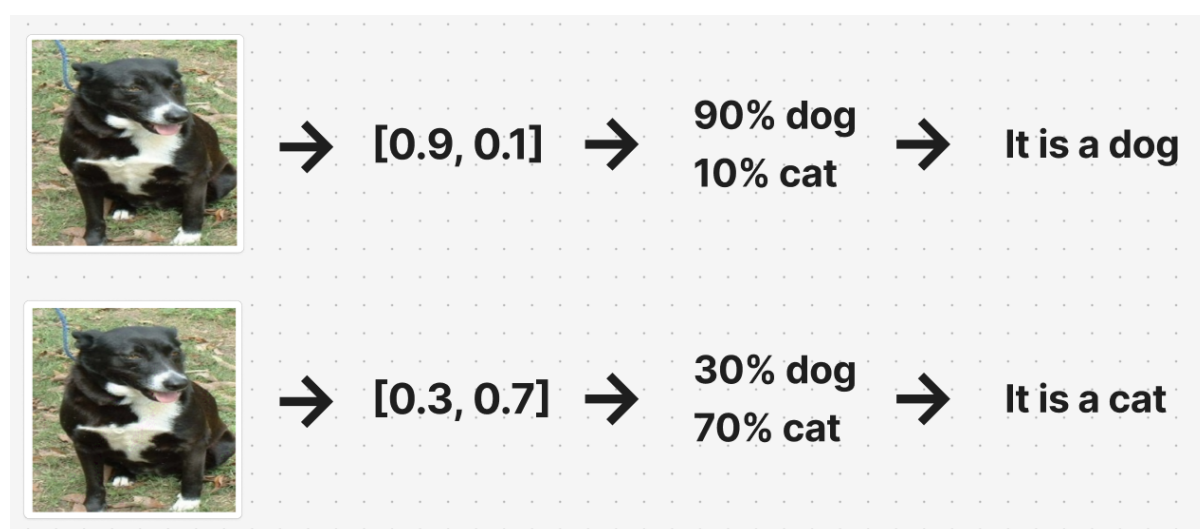


Figure 1.8: Comparison of class probabilities from a CNN for a clean image versus its adversarial counterpart.

The modifications made to the image are extremely subtle and almost imperceptible to the human eye. For this reason, I will deliberately exaggerate them in the illustration below so that you can clearly see which pixels have been altered, as well as the specific color changes that were applied.

This visualization will help you understand how minimal, targeted modifications can

have a major impact on the model’s prediction, even though they may appear insignificant to us (see Figure 1.9).

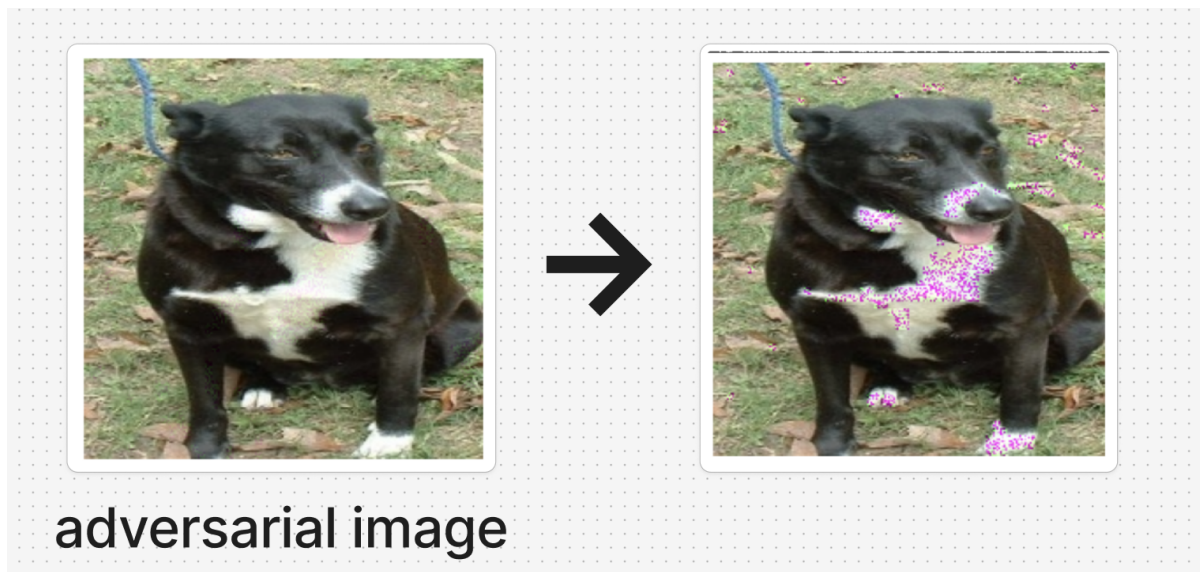


Figure 1.9: Visualization of an adversarial image alongside a version with exaggerated pixel mutations for perceptual clarity.

Before concluding this chapter, we believe it is essential to clarify two key concepts in adversarial machine learning: *targeted attacks* and *untargeted attacks*.

A **targeted attack** refers to a specific type of adversarial manipulation in which the objective is to intentionally steer the model’s prediction toward a predefined label. For instance, if the original image is that of a dog, the goal of the attack may be to have the model misclassify it specifically as a cat. In this case, the adversarial perturbation is carefully crafted to increase the probability of a particular target class, regardless of the original prediction.

In contrast, an **untargeted attack** does not aim for a specific misclassification. Instead, its objective is simply to cause the model to make an incorrect prediction, without regard for which wrong label is ultimately assigned. For example, starting from the same image of a dog, the goal may be to ensure that the model does not classify it as a dog, irrespective of whether it is misclassified as a cat, a car, or any other class.

These two attack paradigms differ in their specificity and difficulty: targeted attacks typically require more precise perturbations, as they must navigate the model’s decision boundaries in a particular direction. Untargeted attacks, while often easier to achieve, may still reveal significant vulnerabilities in the robustness of neural networks.

It is also important to emphasize that all adversarial attacks conducted in this study are strictly **targeted**. That is, in each case, we explicitly define the desired target label

and aim to modify the image in such a way that the model's prediction converges toward that specific class. No untargeted attacks are considered in this paper.

Chapter 2

Edge-Guided Perturbations for Adversarial Image Generation

In this chapter, we introduce a new approach to generate **adversarial images** by focusing on the most visually informative regions of an image, its *edges and contours*. Rather than spreading perturbations across the entire image, we aim to selectively target the areas most likely to influence the model’s prediction. The main idea is simple: By distorting the *structural boundaries* that define an object, we may be able to break the way the model perceives it. Let’s explore how this intuition can be turned into an effective algorithm.

2.1 Submatrix Selection and Edge Detection

To apply our **edge-based strategy**, we first need a systematic way to analyze the structure of an image and identify its most impactful regions. This section describes how we divide the image into *sub-matrices*, detect *edge intensity* using convolutional filters, and score the different regions. These steps are the foundation that will allow us to focus our perturbations more intelligently.

2.1.1 Motivation and Conceptual Intuition

Our first algorithm focuses on the *edges and contours* found within an image. It stems from the following intuitive question: *Is there really a dog in the image if the visual boundaries that separate the animal from its environment are removed or altered?*

After formulating this idea, the next step was to translate it into a practical method.

To do so, we had to break down the problem into smaller components. First, we needed to detect the edges present in the image. Then, we had to find a way to identify the most impactful ones and finally to apply noise to those specific regions.

We began by dividing the image into a list of *sub-matrices*, similar to breaking a puzzle into individual pieces. The size of each sub-matrix was determined by a **divisor** used to split the original image.

This divisor had to be a factor of 224 (the standard dimension of our input images), and the larger the divisor, the smaller the sub-matrices became. This allowed us to control the *granularity* of the analysis by adjusting the fineness with which the image was partitioned (see Figure 2.1).

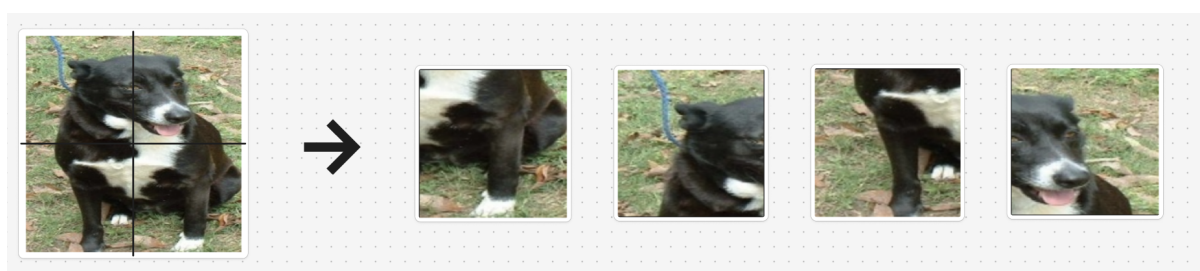


Figure 2.1: Image representation split into four submatrices obtained using a divider value of 2.

In the example above, we chose a divisor of 2 to simplify the explanation. This results in a grid of $2 \times 2 = 4$ sub-matrices, each with dimensions of $224 \div 2 = 112$ pixels, yielding 112×112 regions.

2.1.2 Edge Detection Using Sobel Convolution

At this stage, a natural question arises: *How can we determine which of these sub-matrices contain the most impactful edges?*

While researching this topic, we came across the concept of the **Sobel convolution**, which is commonly used in image processing [3]. When applied to a matrix, the Sobel filter produces a score that reflects the presence of horizontal or vertical edges within that region.

The higher the *absolute value* of this score, the more likely and significant the presence of an edge is in that sub-matrix (see Figure 2.2).

-1	-2	-1
0	0	0
1	2	1

Figure 2.2: Visualization of the horizontal Sobel kernel used for edge detection.

To obtain the *edge intensity scores*, we simply need to apply the Sobel convolution to each sub-matrix. This gives us the absolute score for every region, allowing us to identify and select the top n sub-matrices with the highest edge activity.

However, there is a subtle but important detail to address: the standard Sobel kernel shown above is designed to detect *horizontal edges only*. As such, it fails to capture other types of contours or boundaries—particularly vertical ones.

To address this limitation, I modified the kernel by shifting the row of zeros in the center, thus creating the following.

- A **vertical Sobel kernel**, sensitive to vertical boundaries,
- And two additional **diagonal variants**, each detecting edges with different orientations or emphasis.

These custom kernels extend edge detection capability beyond the standard horizontal filter, and their structures are shown below (see Figure 2.3).

-1	0	1		-2	-1	0		0	1	2
-2	0	2		-1	0	1		-1	0	1
-1	0	1		0	1	2		-2	-1	0

Figure 2.3: Vertical Sobel kernel (left) followed by two diagonal Sobel kernels (right) for gradient-based edge extraction.

It is important to emphasize that the **absolute value** of the convolution score must be used. This ensures that all edges are correctly detected, regardless of their gradient

direction. Without taking the absolute value, strong edges in one direction could be overlooked due to negative values canceling out positive contributions.

Formally, let us denote by M_i the i -th sub-matrix extracted from the original image, and let K_j represent one of the four Sobel kernels introduced earlier (horizontal, vertical, and the two diagonal variants). The edge intensity score S_i of sub-matrix M_i is then computed as the sum of the absolute values of the convolutions with each kernel:

$$S_i = \sum_{j=1}^4 \|M_i * K_j\|_1$$

Here, $*$ denotes the 2D convolution operation and $\|\cdot\|_1$ indicates the sum of absolute values throughout the resulting feature map. This formulation ensures that all edges, regardless of orientation, contribute positively to the final score.

The sub-matrices with the highest S_i values are then selected as the regions most densely populated with edges and contours.

Before moving on, I would also like to highlight an important implementation detail:

In order to achieve the most precise and sensitive results possible, I decided to apply the Sobel convolution *individually to each RGB channel*. This choice significantly increases the total number of sub-matrices we must evaluate, multiplying it by a factor of 3.

For instance, when the divisor is 2 and the original image is split into 4 sub-matrices, this results in $4 \times 3 = 12$ sub-matrices to be tested (4 per channel).

This approach proves especially useful in specific cases, such as when the object of interest is reddish in color, like a ginger dog. In such scenarios, conventional edge detection methods might primarily highlight high-contrast black and white edges, potentially missing crucial contours defined by red intensity.

By treating the red channel separately, the algorithm is able to isolate and emphasize edges that might otherwise be overlooked, and correctly prioritize the corresponding sub-matrix based on its elevated convolution score (see Figure 2.4).

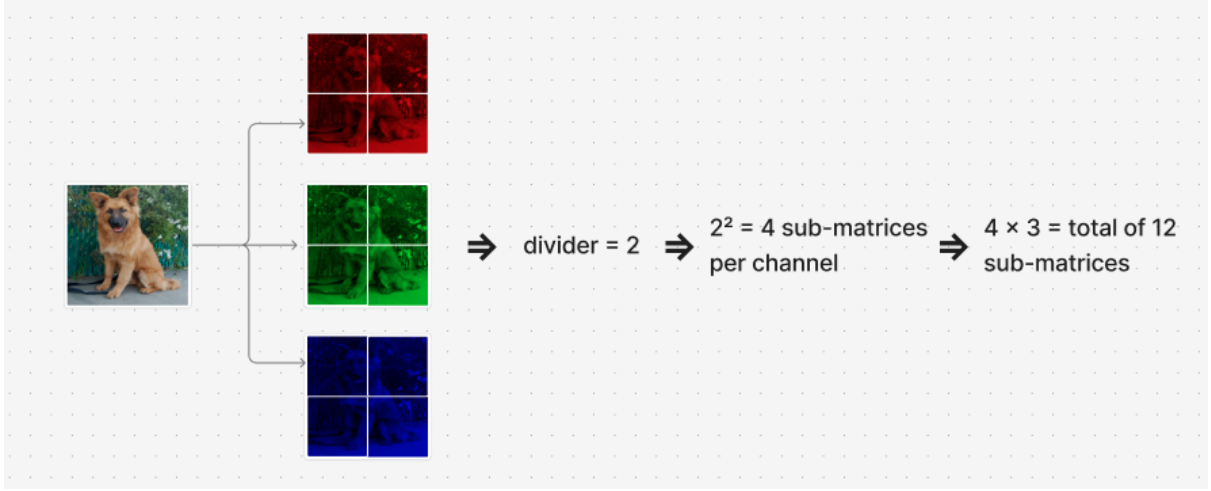


Figure 2.4: Decomposition of an image into 12 sub-matrices, four per channel, using a divider value of 2.

2.1.3 Perturbation Generation and Smoothing

Additionally, it is worth noting that even when each sub-matrix has dimensions of 112×112 , the Sobel kernel remains fixed at a size of 3×3 . This is intentional: in this version of the algorithm, our objective is to identify the sub-matrices that contain the most significant and densely concentrated edges.

In another variant of the algorithm, we explored a different strategy in which the Sobel kernel matches the size of the sub-matrix itself. This allowed us to highlight individual sub-matrices containing a single, highly distinctive edge, rather than aggregating edge activity across a wider region.

Now that we have identified the sub-matrices most likely to contain strong, well-defined, and characteristic boundaries, the next step is to inject noise into these regions.

From this point on, we will refer to the sub-matrices identified as dense in edges and highly impactful, as `elite_matrices`.

In order to apply a subtle and more controlled perturbation, we chose not to treat each `elite_matrix` as an independent region, but instead to group all `elite_matrices` into a single unified region.

This allows us to control the total amount of noise added by, for example, injecting perturbations into only 1% of the pixels within the combined `elite_matrices` region.

To implement this, we first gathered all pixel coordinates contained within the selected `elite_matrices`. From this set, we randomly selected a subset of coordinates according

to a predefined *percentage* parameter, which controls the sparsity of the perturbation across the image.

Subsequently, we applied additive noise to the selected coordinates. The noise values were sampled uniformly at random from the interval $[-x, x]$, where the scalar value x defined by the parameter **reach** determines the maximum amplitude of the perturbation. This allows for precise control over the intensity of the modifications introduced at each pixel location.

2.2 Evolutionary Algorithms

In this section, we introduce a different strategy for generating adversarial images, one that does not rely on predefined regions or deterministic rules. Instead, the algorithm is allowed to explore the search space freely through randomness, selection, and iterative refinement. Inspired by natural evolution, this method simulates mutation, crossover, and survival of the fittest to discover effective perturbations. Our implementation is based on the evolutionary framework developed by Ali Osman Topal, Raluca Chitic, and Franck Leprévost [11], which we adapted to suit the specific goals of this study.

2.2.1 Initialization and Parameterization

In order to find the specific combination of noise that produces a desired confidence score for a given label, we must iterate over multiple noise configurations in the most efficient way possible.

To begin this process, we start by generating n copies of the clean (unmodified) image. Each of these copies will serve as a base instance to which a different combination of noise will be applied.

Next, we apply a different and randomly generated targeted noise to each of the image copies, while respecting a set of predefined parameters. These parameters govern the structure and intensity of the perturbation applied to each instance.

Specifically, we define:

- **reach** : the amplitude of the noise to be injected, which determines how strong the pixel-level modifications can be (see Figure 2.5);
- **elite_matrices** : the number of sub-matrices that will be selected for perturbation—those with the most significant edge scores (see Figure 2.6);

- **divider** : the value used to subdivide the original image, which in turn defines the size and number of sub-matrices (see Figure 2.6);
- **pourcentage** : a parameter that defines the percentage of the total image area to be modified, used to automatically determine the number of **elite_matrices**.

Each copy of the image will receive a unique combination of noise, following the logic defined by these parameters.

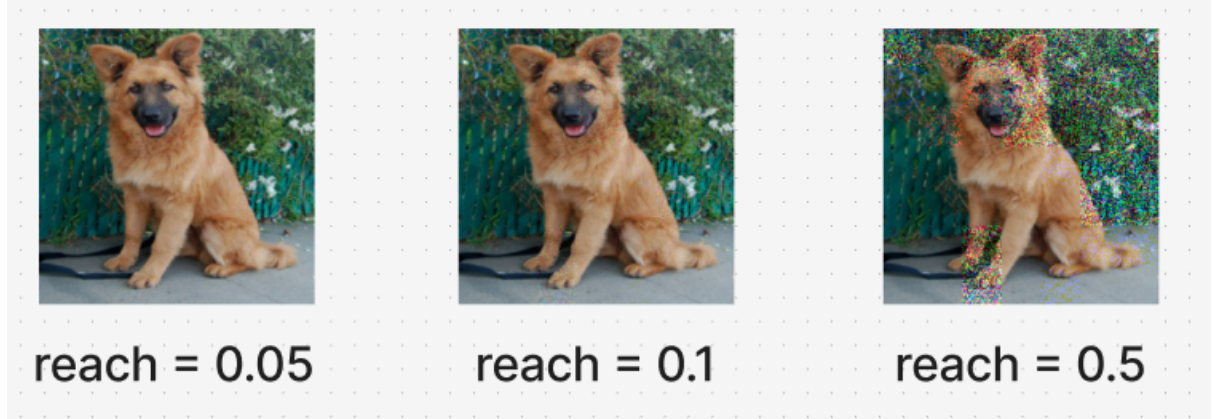


Figure 2.5: Visual impact of the **reach** parameter controlling the saturation of injected noise in adversarial images.

Before moving forward, it is worth making a brief aside on the **divider** parameter, which determines the number and size of sub-matrices and ultimately affects the spatial precision of the noise injection.

As one might expect, the larger the **divider**, the smaller each sub-matrix becomes, which in turn allows the noise to be more finely localized. This results in tighter, more targeted perturbations, capable of affecting highly specific visual structures such as narrow edges or contours.

An extreme yet highly effective example would be a divider of 56, which splits a 224×224 image into $56 \times 56 = 3,136$ sub-matrices, each of size 4×4 . In this configuration, the noise can be applied with pixel-level granularity, allowing it to precisely contour the critical boundaries previously identified in the edge detection phase (see Figure 2.6).

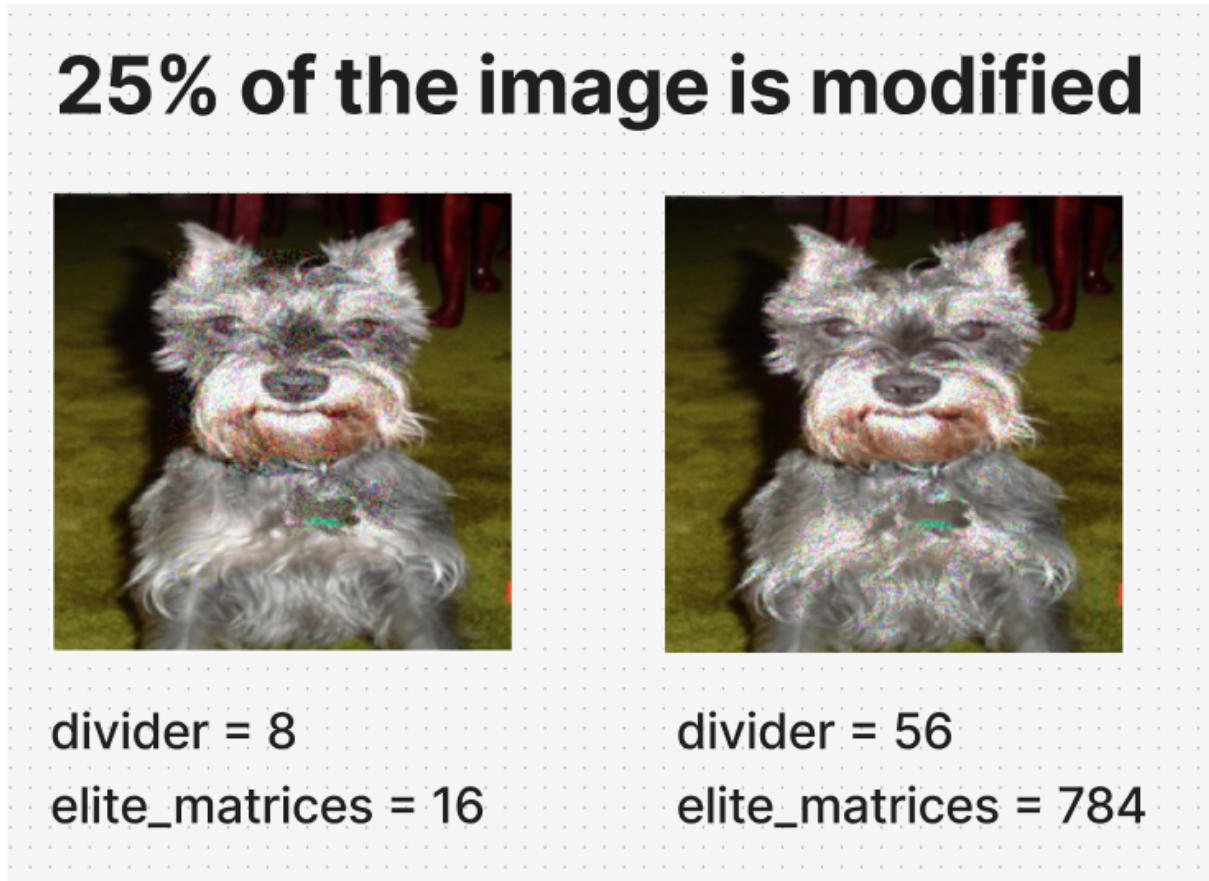


Figure 2.6: Effect of increasing the `divider` parameter on the precision of noise localization, with a constant noise ratio of 25%.

2.2.2 Evaluation and Selection Strategy

In order to efficiently filter and improve solutions across successive iterations, and with the goal of maximizing performance, we naturally chose to use an **evolutionary algorithm**.

However, we are aware that not all readers may be familiar with this concept. For that reason, I will take a brief moment to explain the underlying principle and mechanism behind evolutionary algorithms, so that you can follow the rest of the methodology with clarity.

An *evolutionary algorithm* is an optimization technique inspired by the principles of natural evolution. It begins by generating an initial set of random solutions to a given problem, similar to a population of individuals which, in our case, corresponds to the modified images we have created.

Each solution is then evaluated on the basis of its performance, and the best ones are selected. These “surviving” individuals are altered, combined, or slightly perturbed to form a new generation.

This process is repeated over multiple generations, gradually leading to progressively better solutions.

Evolutionary algorithms are especially powerful when exploring large and complex search spaces, making them a particularly effective tool for generating adversarial images that can fool sophisticated machine learning models such as the **CNN** used in this study [5, 1, 11].

Now that the fundamental principles of evolutionary algorithms have been introduced, and considering that we have already generated a batch of modified images, each containing targeted, edge-based perturbations, it is necessary to perform a selection step. Specifically, we identify an *elite population*, defined as the subset of individuals within the current batch that achieve the highest prediction scores for the targeted label. This selection process ensures that only the most promising candidates are retained for the subsequent iteration of the algorithm.

- One quarter of the batch is selected as the **elite group**;
- One quarter is assigned to the **middle group**;
- The remaining half of the batch constitutes the **low-performing group**.

For example, if we start with a batch of 40 modified images, we will select the top 10 performing images as our elite set.

The remaining 30 images are then divided as follows:

The next 10 best-performing images (ranked just below the elite set) form a middle tier;

The remaining 20 images are categorized as the lower tier, but they are not used in this version of the algorithm.

The figure below illustrates the population structure and selection process (see Figure 2.7).

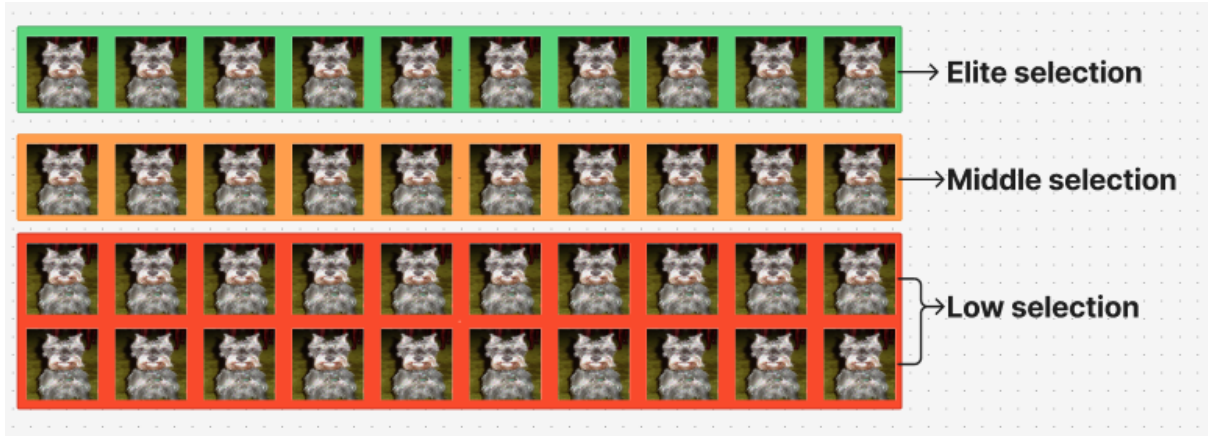


Figure 2.7: Illustration of the subdivision into elite, middle, and low selection groups based on edge scores.

2.2.3 Fitness Scoring with CNN Predictions

To determine whether a given image has good performance or not, we pass the entire set, also called the **batch** of modified images through our **CNN** model.

This process yields, for each image, a vector of probabilities of *classes*, representing the probability that CNN assigns to each of the possible labels.

From these outputs, we extract the probability associated with the specific *target label*, for example, the probability that the image is classified as a cat.

This gives us a single score per image, corresponding to how strongly the CNN is "convinced" that the perturbed image belongs to the target class.

We then sort these scores in ascending order, allowing us to easily identify the most successful perturbations, the ones that most effectively push the CNN toward the desired misclassification (see Figure 2.8).



Figure 2.8: Grouping of images based on CNN predictions, highlighting internal divisions by output class likelihood.

2.2.4 Crossover and Iterative Evolution

To efficiently update the image batch at each iteration, we follow the process below before evaluating the new generation.

1. **Elite preservation:** The elite selection is set aside and directly contributes 10 images to the new batch, without modification.
2. **Middle mutation:** The middle selection contributes 10 images to the new batch. Each of these images is mutated once before being added.
3. **Random recombination and double mutation:**
 - A set of 20 images is formed by combining the elite selection with 10 randomly chosen images from the previous batch.
 - Each of these 20 images undergoes two consecutive mutations to increase variation.
4. **Mutation operation:** Mutation consists of adding controlled noise to a specific region of the image. The strength and area of the perturbation are defined by a percentage of the targeted region.
5. **Evaluation:** The new batch of 40 images is then evaluated to determine whether a high-performing adversarial image has been discovered.

It is also important to mention that, after each mutation, the modified images are *clamped* to remain close to their original pixel values. This step helps to control the noise and maintain subtle perturbations. For example, if a pixel initially has a value of 0.5 and the `reach` parameter is set to 0.1, the pixel will be restricted within the range $[0.4, 0.6]$.

Now that the images have been ranked and grouped, the next step is to generate improved candidates by reusing elements from existing ones. To do this efficiently, we apply a method known as *crossover*.

Let us consider an image from the elite group, one that CNN is more likely to misclassify as a cat, despite it clearly being a dog to the human eye.

The challenge lies in the fact that we do not know which specific region of the image is responsible for the misclassification. It could result from modifications in the upper right corner, the lower left, or even from a combination of subtle perturbations distributed across multiple areas of the image. Without direct insight into the model’s internal decision-making process, identifying these critical regions becomes inherently difficult.

This uncertainty exists for all elite images. That is why we apply a technique known as *crossover*, where we randomly extract a region from one elite image and combine it with another (see Figure 2.9).

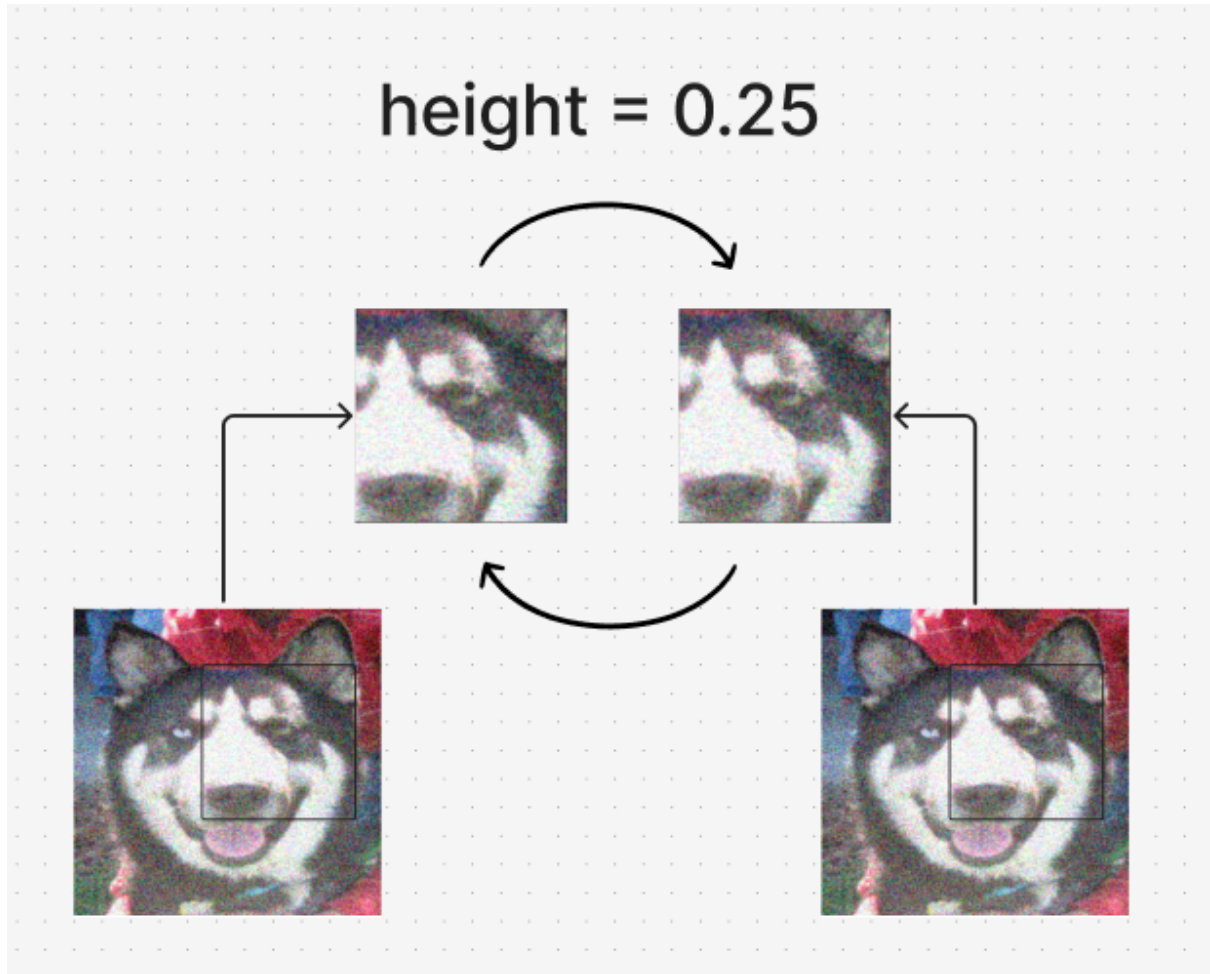


Figure 2.9: Crossover operation between two mutated images, restricted to a 25% region of the image area.

The only parameter that determines the size of the region exchanged during the crossover is a variable called `height` in our algorithm. This parameter can take a value between 0 and 1 and defines the size of the exchanged section as a percentage of the original image.

For instance, if `height` is set to 0.15, this means that the region to be swapped represents 15% of the total image area.

In our implementation, crossover is applied exclusively to the images that were mutated in the previous step, that is, to all images except those in the elite selection.

This means that three-quarters of the batch (30 out of 40 images) are eligible for crossover.

To ensure that cross-over occurs between randomly selected images, we randomly sampled and group the images into pairs, each of which will exchange portions of their pixel data.

This randomness in pairing helps prevent convergence toward repetitive patterns and promotes greater diversity in the generated adversarial examples.

Finally, once the crossover has been completed, the newly formed batch, composed of the unchanged elite selection and the images resulting from crossover and mutation, is passed through the CNN.

This allows us to obtain updated predictions for each image and determine whether new high-performing adversarial examples have been successfully discovered.

This cycle is repeated until we obtain an image that is sufficiently effective, that is, one for which the CNN assigns a high probability to the target label.

2.3 Experimental Objective and Setup

Now that the algorithm has been fully defined and its components have been carefully explained, it is time to test its effectiveness in the real world. In this section, we design and execute a series of experiments to evaluate whether our method holds in practice. The goal is simple: determine whether targeting specific visual structures, such as edges and contours, actually improves the speed or quality of adversarial image generation compared to more naive strategies. Each experiment has been carefully designed to isolate specific parameters and evaluate their impact with clear and consistent metrics.

2.3.1 Experimental Configuration

All experiments described in this study were conducted on the same hardware and software setup to ensure consistency and reproducibility. The implementation used for all experiments is available on the GitHub repository: https://github.com/trql/EA_pytorch.

The machine used is equipped with an Intel i5-10400F CPU, 16 GB of DDR4 RAM clocked at 3200 MHz, and an NVIDIA RTX 3060 GPU with 12 GB of VRAM. The operating system was Ubuntu 24.04 LTS, and all code was written and executed using Python 3.12.9 and PyTorch 2.5.1, along with standard libraries such as `torchvision`, `timm`, and `matplotlib`.

Throughout this work, we used the **ResNet-50 convolutional neural network**, a widely adopted model known for its balance between simplicity, performance, and robustness. Unless otherwise stated, this CNN architecture remained unchanged across all experiments.

The key hyperparameters were kept constant in all experiments to ensure stability and facilitate direct comparisons (see Figure 2.10).

```
# STABLE PARAMETERS

epochs = 10000 # number of iterations

batch = 40 # number of images in a batch

wanted_class = 285 # targeted class

pourcentage_fixed = 0.01 # pourcentage of pixels modified per iteration

height = 0.15 # size of the crossover zone

reach = 0.03 # amplitude of the added noise
```

Figure 2.10: Constant hyperparameters used across all experiments to ensure stability and comparability.

These parameters were initially selected based on empirical performance in preliminary tests involving random noise injection, and they proved to provide a strong baseline for subsequent analyses [11]. Throughout the experimental chapters, specific parameters will be varied in isolation to assess their individual impact on adversarial success.

Unless otherwise noted, all adversarial examples were generated using dog images as the source class. We targeted class 285, labeled “Egyptian Cat,” as our adversarial objective. This class was chosen for its moderate visual and semantic similarity to the source images, which helps avoid overly unrealistic scenarios and supports more meaningful evaluation. To establish a reference baseline, we present the top predictions made by ResNet-50 for the ten original 10 dog images from the *Microsoft Cat vs Dog Dataset* [4] used throughout the study (see Table 2.1).

Image	Best Prediction	Best Label
0	0.591	227
1	0.827	209
2	0.295	172
3	0.496	207
4	0.995	251
5	0.657	196
6	0.962	678
7	0.452	235
8	0.841	249
9	0.414	250

Table 2.1: Best predictions and labels before modification for each image.

2.3.2 Objective of the Experiment

We must first define what constitutes success in our context.

We will consider the algorithm successful in the following cases:

- If it achieves **better results** than the baseline;
- If it achieves **similar results in fewer iterations**, by specifically targeting regions with a high concentration of edges;
- If it achieves **worse results than random noise**, as this would provide evidence *against* the hypothesis that CNNs base their decisions primarily on detailed areas containing corners and edges.

In the last case, the outcome would still be valuable, as it would challenge the hypothesis that CNNs rely heavily on detailed regions with high concentrations of edges and corners to make their predictions [12].

Weaker performance of the targeted method would provide evidence against this assumption, suggesting that such visually structured regions do not significantly influence the model’s classification decisions.

2.3.3 Execution Time Benchmarking

All experiments in this chapter were conducted using the same hardware and software configuration detailed in subsection 2.3.1. The CNN architecture, hyperparameters, and image inputs remained identical across experiments to ensure fair comparison and reproducibility.

However, since the internal logic and computational complexity of each adversarial generation algorithm vary from chapter to chapter, the runtime per epoch may differ significantly.

To provide a concrete reference for performance evaluation, we report below the average execution time (in seconds) required to complete 100, 1,000, and 10,000 epochs for the algorithm implemented in this chapter:

- **100 epochs:** 78 seconds
- **1,000 epochs:** 763 seconds
- **10,000 epochs:** 6746 seconds

These measurements were obtained by running the full training loop on a fixed input batch of 40 images targeting class 285 (Egyptian Cat), using the standard hyperparameters and a value of `min_pourcentage` equal to 1. They serve as a useful metric for assessing the scalability and efficiency of the method in practical use cases.

2.3.4 First Experience

As you may know, experiments in this domain are often computationally expensive and resource-intensive.

To make the first phase of testing lighter yet meaningful, we decided to test all possible combinations between a list of `divider` values and a list of `min_pourcentage` values (see Figure 2.11).

While the parameters `manual` and `elite_matrices` can be used to manually specify the exact number of sub-matrices to modify, in our case we will rely solely on the parameter `min_pourcentage`.

This parameter defines the number of sub-matrices to be perturbed as a percentage of the total image area. For instance, if the image is divided into 4 sub-matrices and `min_pourcentage` is set to 50%, then `elite_matrices` will automatically be set to 2.

```

# CHANGING PARAMETERS

changing_divider = [8, 16, 32, 56]

changing_min_pourcentage = [
    6/64, # 0.09375 ~ 0.1
    13/64, # 0.203125 ~ 0.2
    26/64, # 0.40625 ~ 0.4
    39/64, # 0.609375 ~ 0.6
    51/64, # 0.796875 ~ 0.8
    57/64, # 0.890625 ~ 0.9
    61/64, # 0.953125 ~ 0.95
    64/64 # 1.0
]

```

Figure 2.11: Valeurs testées pour les paramètres `divider` et `min_percentage` lors de la première expérience.

Each configuration is evaluated over only 100 epochs, and we track the evolution of the target label’s predicted probability throughout the optimization process.

The main objective of this experiment is to identify which combinations of parameters lead to faster convergence toward a successful adversarial outcome. These early results will already give us insightful directions to prioritize in subsequent experiments with more epochs.

To ensure that the findings are statistically meaningful, we repeat each configuration 10 times, and conduct the tests on 10 different input images, ensuring that the results are not biased by any one particular image or isolated case.

Before continuing with the experiment, there are three important clarifications worth noting.

1. A value of `min_percentage` = 1.0 is **equivalent to random noise injection**, since 100% of the image is selected. In this case, the perturbation is no longer targeted, and the behavior is essentially the same as a non-guided adversarial attack.
2. The **intensity of the added noise is adapted** to the size of the affected area. For example, adding noise to 1% of the pixels across 100% of the image is equivalent to adding noise to 2% of the pixels across 50% of the image. This ensures that the overall modification rate remains constant across different configurations, enabling fair comparisons.
3. While the values of `min_percentage` may seem arbitrary, they were carefully selected so that they correspond to an **exact integer number of elite_matrices**.

For instance, with `divider` = 8, we obtain $8 \times 8 = 64$ sub-matrices. A value of `min_pourcentage` = 0.9 gives $64 \times 0.9 = 57.6$, which rounds to 58. This introduces small inaccuracies. Our chosen values avoid this rounding by design, ensuring precise proportions and unbiased results.

The table below presents the 10 best performing parameter combinations across the 10 tested images. These results highlight the most effective configurations identified during our initial exploration phase (see Table 2.2).

Image	Divider	Min Pourcentage	Evolution of the Prediction
0	56	1.00	0.00117
1	8	1.00	0.00233
2	32	0.891	0.000576
3	56	0.953	0.000824
4	32	0.609	1.64×10^{-5}
5	8	0.406	8.66×10^{-5}
6	32	0.797	6.05×10^{-5}
7	8	1.00	0.00158
8	8	0.953	8.27×10^{-6}
9	8	0.797	6.75×10^{-5}

Table 2.2: Best-performing combinations of `divider` and `min_pourcentage` per image, achieving optimal convergence toward the target label within the first 100 epochs.

The results are quite surprising, as we observe that the best-performing configurations often correspond to a `min_pourcentage` between 70% and 90%. This suggests that while targeting only the most sensitive areas may not always be the most effective strategy, avoiding the least sensitive regions might actually be a good approach.

2.3.5 Second Experience

To investigate this further, we will conduct an additional experiment focusing on configurations that used a `min_percentage` strictly different from 1. This time, we will increase the number of epochs to 1000 and compare each of these configurations to their untar-geted, fully random alternative with `min_percentage` = 1. This comparison will allow us to determine whether targeting dense areas rich in edges and corners is genuinely more effective than broad and imprecise modifications.

As before, we will repeat this experiment 10 times to calculate the average results and ensure their reliability (see Table 2.3).

Image	Prediction Targeted	Prediction Untargeted
2	0.0228	0.0142
3	0.0122	0.0127
4	0.000527	0.000733
5	0.000413	0.000644
6	0.00158	0.00269
8	0.00217	0.00418
9	0.00928	0.0147

Table 2.3: Comparison of prediction confidence for the target label over 1000 epochs between the best-performing targeted and untargeted strategies per image.

As we can see, in the vast majority cases, modifying 100% of the image is much more effective than altering only a reduced percentage, even if that reduced area is richer in edges and corners.

However, this comparison is not entirely fair; we are not comparing apples to apples. Indeed, beyond a certain number of epochs, the version of the image modified at 100% is guaranteed to accumulate more total noise than a more targeted version, simply because the noise is bounded and cannot exceed the range $[-\text{reach}, +\text{reach}]$.

On the other hand, a closer look at the results of the first experiment actually reflects positively on our algorithm. When total noise quantity is not the limiting factor, as in the first experiment with only 100 epochs, targeting the most strategic areas of the edges seems to be a good strategy. This is demonstrated by the speed at which results improved in just 100 epochs.

To definitively settle this matter, a third experiment is necessary, one that enables a truly fair comparison. We will reproduce Experiment 2, but instead of comparing it against the version of the algorithm that modifies 100% of the image, we will compare it to a version that modifies the *same percentage* of the image—but selects its sub-matrices at random. In this variant, `elite_matrices` are not selected based on edge density but are instead randomly sampled from all available sub-matrices.

This will allow us to finally determine whether the strategy of targeting edge-dense regions is genuinely effective, or whether its performance is comparable to random selection (see Table 2.4).

Image	Prediction Targeted	Prediction Untargeted
2	0.0239	0.0125
3	0.0121	0.0119
4	0.000458	0.000317
5	0.000452	0.000291
6	0.00235	0.000989
8	0.00318	0.00261
9	0.00823	0.00323

Table 2.4: Target label prediction comparison over 1000 epochs between targeted and untargeted algorithms, under a fixed pixel modification ratio.

As we can observe, targeting edge-dense regions is in fact an effective strategy, as it consistently yields higher probabilities in 100% of the cases, even when the experiment is repeated ten times. On average, focusing on strategic zones is **1.7 times as effective** as targeting random zones.

However, we also note that not all images respond in the same way. For some images, the difference between targeting random zones and targeting strategic zones is relatively small and only slightly reflected in the resulting probabilities. This can be explained by the fact that, while our algorithm does indeed target the most edge-dense regions, it does not guarantee that these regions overlap with the main object of interest in the image, such as the dog or cat depicted.

If, for instance, the portion of the image being modified is relatively small (e.g., 10%), and the background contains a high density of edges, then the algorithm may appear highly ineffective because it may not directly modify the subject of the image.

In conclusion, we can affirm that, when compared fairly, targeted modification is a valid and powerful strategy. On average, it performs **1.7 times as well** as a random modification and proves particularly effective in adversarial image generation scenarios where the prerequisite is to modify only a limited percentage of the image.

2.3.6 Conclusion

This series of experiments was designed to rigorously evaluate the effectiveness of our proposed edge-targeted adversarial image generation algorithm. The results allow us to draw several key conclusions.

First, when compared to fully random perturbations over the entire image, our algorithm performs comparably or even slightly worse in longer runs. This suggests that the

quantity of noise injected over time plays a major role in adversarial success, particularly in scenarios where modification constraints are relaxed.

However, a more nuanced picture emerges when we restrict the amount of noise injected and evaluate short-term performance. Under these conditions, limited noise budget and reduced number of epochs, our targeted approach clearly outperforms random selection. The ability to focus perturbations on high-gradient regions yields faster convergence and more significant prediction shifts. This finding strongly supports the hypothesis that convolutional neural networks are more sensitive to localized modifications around edges and contours.

Crucially, in the final experiment, where we fixed the total noise and compared targeted versus untargeted sub-matrix selection under identical constraints, the edge-focused strategy proved superior in all tested cases. On average, it was 1.7 times more effective at driving the model toward the target class. This consistent advantage confirms the validity of using structural cues, such as edge density, as a means of guiding adversarial optimization.

Nevertheless, we also observed variability across different images. In some cases, the improvement was marginal, likely due to the misalignment between high-edge-density regions and the actual location of the semantic object within the image. This highlights a potential limitation of our method: targeting edges alone does not guarantee semantic relevance.

In summary, while our approach does not universally outperform untargeted methods in all scenarios, it provides a clear benefit when working under constrained perturbation budgets. These findings not only validate the intuition behind structured adversarial generation but also suggest promising avenues for refinement, such as integrating object localization techniques to enhance spatial alignment between perturbations and critical semantic regions.

Chapter 3

Evolutionary Generation of Universal Adversarial Masks

In the previous chapter, we explored how adversarial examples can be generated by targeting specific regions of an image, such as edges and contours. In this new chapter, we shift our focus toward an entirely different challenge: generating a single noise pattern that can be applied to any image and still succeed in fooling a **CNN**. This type of perturbation, known as a *adversarial mask*, pushes the limits of what adversarial attacks can achieve by removing all dependency on the image’s content.

3.1 Universal Adversarial Mask Generation

In this section, we introduce the core idea behind universal adversarial masks: noise patterns designed to fool a CNN into predicting a specific label regardless of the input image. Instead of customizing perturbations for each image, our goal is to construct a standalone, transferable mask that can be blended into any image with minimal adjustment. We describe the motivation, construction process, and blending strategy that make this possible.

3.1.1 Motivation and Noise Construction

In our pursuit of generating the most effective adversarial images as efficiently as possible, we developed several well-founded and meaningful algorithms, such as the edge detection-based strategy described in the previous chapter.

However, we also designed a surprising and novel approach that aims to generate **uni-**

versal adversarial masks—perturbations that can be applied to any image, regardless of its content, in order to mislead a CNN into predicting a specific label.

We already had all the algorithms required to generate adversarial images from a given input image using random, non-targeted noise.

Therefore, the only adjustment needed was to replace the input image with a neutral tensor. Instead of providing a real image to the evolutionary algorithm, we supplied a uniform tensor of size 224×224 filled with the value assigned to the **reach** parameter.

We initialize the tensor with a constant value equal to **reach**, ensuring that the **minimum possible value remains 0**, since the noise added will always range between **-reach** and **+reach**.

The objective: to allow the algorithm to discover a noise pattern that, while visually meaningless to humans, would be classified by the CNN as a specific label with high confidence.

The parameter **reach** controls the amplitude of the perturbation and cannot be greater than 0.5, to ensure that no pixel value exceeds the upper bound of 1.0, which is the maximum intensity of a pixel in PyTorch.

The result is a fully random noise tensor that carries no semantic information, yet is shaped in a way that can potentially deceive a CNN classifier.

For example, the following mask represents a **great white shark** (label 3), generated with a noise amplitude of **reach = 0.03**, and is classified by the **ResNet-50 CNN** with a **92% confidence score** (see Figure 3.1).

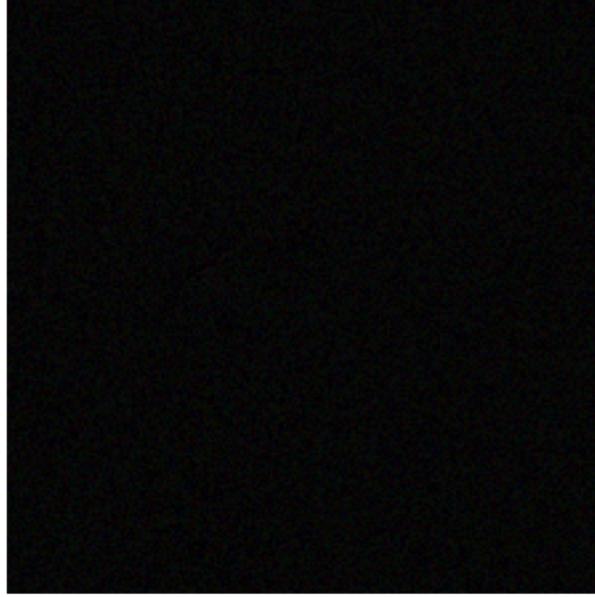


Figure 3.1: Adversarial noise pattern classified as a great white shark (class 285) with 92% confidence.

3.1.2 Mask Application via Alpha Blending

Now that we have an effective method for generating adversarial masks, the next challenge was to find a way to apply these masks to real images to successfully attack the CNN.

To do this, we use a technique known as *alpha blending*.

Alpha blending is a common method in image processing that allows two images (or tensors, in our case) to be combined based on a weighting factor, usually referred to as **alpha** [2]. This factor determines how much influence each image should have in the final result.

For example, if we want the mask to contribute 50% of the intensity, we simply divide both the mask and the image by 2 before adding them together. This ensures that the final blended pixel values do not exceed 1.0, which is the maximum valid pixel intensity in PyTorch.

In a more general case, suppose we want the mask to contribute 30% of the final intensity. In this case, we multiply the mask by 0.3 and the original image by $1 - 0.3 = 0.7$, and then sum the two results. Again, this guarantees that pixel values remain within the valid range $[0, 1]$.

Now that you understand how alpha blending works, we can explain how we apply our generated masks to input images.

In our implementation, we aim to preserve the adversarial noise that we have generated and simply *add it to the image* by making the necessary space for it in the pixel values.

Since the adversarial noise is generated with a **reach** parameter, its values range from 0 to $2 \times \text{reach}$. This is because the noise tensor is initialized with a base value of **reach**, and then a random perturbation between $-\text{reach}$ and $+\text{reach}$ is added to each element.

To ensure that there is enough room to add the noise without exceeding the valid pixel range $[0, 1]$, we scale the original image by $1 - 2 \times \text{reach}$ before adding the noise.

This approach guarantees two key properties:

- The **adversarial mask remains unaltered**, preserving its effectiveness;
- The resulting image tensor remains within the **valid intensity bounds**, ensuring compatibility with the CNN.

3.2 Experimental Objective and Setup

In this section, we put our universal adversarial mask generation strategy to the test. While previous experiments focused on crafting image-specific perturbations, our goal now is to determine whether a single, image-agnostic mask can consistently influence the predictions of a CNN model.

We structure our investigation into two main experiments: the first evaluates whether such masks can be successfully generated across different noise amplitudes, while the second assesses their *transferability*, testing whether a mask optimized in isolation can effectively perturb multiple distinct input images. These tests aim to provide a deeper understanding of the strengths and limitations of universal adversarial perturbations.

3.2.1 Objective of the Experiment

We begin by clarifying the precise objective of the upcoming experiments.

These experiments aim to evaluate whether it is possible to generate **universal adversarial masks** using our evolutionary algorithm that remain effective across different input images, consistently pushing the CNN’s predictions beyond a predefined confidence threshold.

In the ideal case, we would be able to produce a mask that is sufficiently effective such that there is no longer any need to create image-specific adversarial examples. This

would represent a major step forward, offering both a significant time-saving advantage and a valuable methodological discovery.

We are fully aware that the predictions obtained by applying the mask to a real image may not match those obtained from the mask alone. However, that is not the goal of this experiment.

Our true objective is to determine whether it is possible to significantly shift the CNN’s prediction in a specific direction using only a *minimally perceptible mask*, regardless of the underlying image.

3.2.2 Execution Time Benchmarking

All experiments in this chapter were conducted using the same hardware and software configuration detailed in subsection 2.3.1. The CNN architecture, hyperparameters, and image inputs remained identical across experiments to ensure fair comparison and reproducibility.

However, since the internal logic and computational complexity of each adversarial generation algorithm vary from chapter to chapter, the runtime per epoch may differ significantly.

To provide a concrete reference for performance evaluation, we report below the average execution time (in seconds) required to complete 100, 1,000, and 10,000 epochs for the algorithm implemented in this chapter:

- **100 epochs:** 99 seconds
- **1,000 epochs:** 941 seconds
- **10,000 epochs:** 7632 seconds

These measurements were obtained by running the full training loop on a fixed input batch of 40 images targeting class 285 (Egyptian Cat), using the standard hyperparameters. They serve as a useful metric for assessing the scalability and efficiency of the method in practical use cases.

3.2.3 First Experience

To evaluate the effectiveness of this new masking-based adversarial method, we conducted a comprehensive experimental test, following the same methodology as in Chapter 2.

In order to obtain meaningful and reliable results, we designed a carefully structured experiment.

To achieve this, we tried to generate 10 adversarial masks for each tested **reach** value (see Figure 3.2).

We ensured that none of the **reach** values exceeded 0.1, so the resulting masks remain subtle and visually imperceptible.

```
reach = [0.01, 0.03, 0.05, 0.07, 0.09]
```

Figure 3.2: Range of **reach** values explored during the initial experiment to assess noise saturation effects.

The following parameters were used during this experiment. They have already proven their effectiveness in multiple previous trials (see Figure 3.3).

```
# STABLE PARAMETERS
epochs = 10000
batch = 40
wanted_class = 285
height = 0.15
pourcentage = 0.01
targeted_pourcentage = 0.9
```

Figure 3.3: Overview of fixed and stable hyperparameters used throughout the experiments.

The objective of this experiment was two-fold. First, we sought to determine whether it was possible to generate effective adversarial masks using various values of the **reach** parameter. Each generated mask was intended to induce the ResNet-50 model to classify the modified image as class 285 (*Egyptian Cat*) with at least 90% confidence. This specific target label was chosen due to its moderate semantic similarity to the true classes of the base dog images, thereby reducing potential bias introduced by extreme class disparity.

The second goal was to identify a value of **reach** that was both sufficiently effective and consistently impactful, making it a strong candidate for subsequent experiments. To this end, we computed the average of the highest confidence score obtained among the 10 generated masks for each **reach** value, allowing us to identify the most promising parameter configuration (see Table 3.1).

Reach	Mask Prediction Confidence
0.01	0.491
0.03	0.00125
0.05	0.908
0.07	0.901
0.09	0.904

Table 3.1: Top-performing masks for class 285, identified across various **reach** values over 10,000 epochs.

As we can observe, only the values of **reach** above 0.05 appear to be effective.

It is particularly striking to note the significant gap between the results for **reach** = 0.03 and **reach** = 0.05.

Consequently, a good compromise would be to select a value of **reach** = 0.07, as it is safely distant from the instability observed at 0.03. Although **reach** = 0.05 shows decent results in 10 attempts, it remains dangerously close to the failure point.

Therefore, we will use a **reach** value of 0.07 for our subsequent experiments.

3.2.4 Second Experience

Our second experiment dives deeper into the core of the problem. We generated 10 adversarial masks using the same parameters as in the previous experiment. These masks were also optimized to be classified as label 285 (*Egyptian Cat*) with a target confidence of 90%, using a fixed **reach** value of 0.07.

Then each of the 10 masks was applied to 10 distinct base images. Afterward, we collected all the resulting prediction scores and computed the average predicted confidence for each image. This yielded 10 values reflecting the effectiveness of applying an adversarial mask across different inputs (see Table 3.2).

Image	Predicted Probability After Masking
0	4.30×10^{-5}
1	3.50×10^{-4}
2	1.91×10^{-6}
3	3.00×10^{-4}
4	3.40×10^{-6}
5	1.07×10^{-6}
6	2.58×10^{-8}
7	2.50×10^{-4}
8	1.07×10^{-7}
9	8.71×10^{-7}

Table 3.2: Predicted confidence scores for class 285 across different images after applying universal adversarial masks optimized for that label.

As we can observe, the results are uniformly poor: adversarial masks failed to consistently steer the CNN toward the target class. This strongly suggests that generating a simple universal mask optimized for one label is not, on its own, an effective attack strategy.

However, before drawing any definitive conclusions, we consider a follow-up question: Even if the masks did not successfully induce misclassification toward the target label, did they at least manage to significantly distort the original CNN prediction? In other words, did the adversarial mask cause a change in the top predicted class, regardless of the target label?

If so, this would imply that the attack, while not successful in a targeted sense, was still effective as an *untargeted attack*. In such a case, the perturbation would have disrupted the original classification, even if it failed to steer the prediction toward the intended class.

To answer this, we repeated the experiment, but this time we monitored the model’s top-1 prediction before and after the mask was applied (see Table 3.3).

Best Prediction Before Modification	Best Label Before Modification	Best Prediction After Modification	Best Label After Modification
0.591	227	0.742	264
0.827	209	0.781	209
0.295	172	0.541	171
0.496	207	0.183	260
0.995	251	0.941	251
0.657	196	0.921	196
0.962	678	0.995	678
0.452	235	0.397	235
0.841	249	0.433	249
0.414	250	0.459	250

Table 3.3: Comparison of top-1 predicted labels before and after applying a 90%-confidence adversarial mask for class 285.

As shown, the top predicted label changed in **3 out of 10** cases. Additionally, in one instance (image 2), the model’s confidence in the correct class was significantly reduced—from 0.841 to 0.433.

However, in several other cases, the adversarial mask had the opposite effect: it increased the model’s confidence in the original prediction. For example, in image 6, the confidence in label 196 increased from 0.657 to 0.921 after applying the mask.

This phenomenon may be explained by a spatial misalignment between the sensitive regions of the adversarial mask and the main object in the image. If, for example, the most influential components of the mask are centered, but the object of interest is off-center, the mask may end up perturbing primarily the background. This could have the unintended consequence of reinforcing the model’s original prediction by leaving the actual object mostly unaltered.

The universal masks we generated were largely unsuccessful in pushing the model toward the desired class. However, they disrupted the prediction of the top-1 in 30% of the tested cases, which, although not sufficient for practical use, represents a non-trivial result.

3.2.5 Conclusion

In this section, we set out to evaluate the feasibility and effectiveness of universal adversarial masks—perturbations that are independent of specific input images yet capable of

influencing the predictions of a convolutional neural network.

Our results demonstrate that while it is indeed possible to generate masks that achieve high classification confidence when evaluated in isolation, their ability to transfer to real-world inputs remains limited. Specifically, even the best-performing masks failed to consistently steer the model’s predictions toward the target label when applied to distinct base images. This indicates that such perturbations lack the robustness and generalization required for targeted universal attacks.

Nevertheless, our untargeted evaluation reveals a more nuanced outcome. Although the masks were unsuccessful in producing consistent misclassifications toward the target class, they did alter the model’s top-1 prediction in approximately 30% of the tested cases. This disruption, albeit limited, confirms that the masks introduce nontrivial perturbations to the model’s internal representations.

These findings suggest that universal adversarial masks, as implemented here, are not yet viable substitutes for image-specific attacks. However, they highlight the potential of such approaches and motivate further exploration. Future work could focus on improving the alignment between the perturbation and salient image regions, perhaps through adaptive mask generation or by leveraging data-driven initialization strategies. Such improvements may enhance the transferability and effectiveness of universal perturbations, ultimately making them a more practical adversarial tool.

Chapter 4

Adversarial Image Generation in the YCbCr Color Space

In this chapter, we explore a new research direction: generating adversarial images in an alternative color representation. Until now, all perturbations have been applied directly in the **RGB** space, the standard format used in most vision pipelines. However, **RGB** may not be the most optimal space for subtle perturbations.

Inspired by human visual perception and compression standards like *JPEG*, we investigate the **YCbCr** color space. This encoding separates **luminance** from **chrominance**, which opens the door to more *imperceptible attacks*. Our objective is to determine whether applying perturbations in **YCbCr**, particularly in the chrominance channels, can offer a better trade-off between *stealth* and *adversarial effectiveness*.

4.1 Color Space Transformation and Algorithm Integration

Before evaluating the effects of perturbations on **YCbCr**, we must first integrate this new color space into our adversarial image generation pipeline. This involves implementing a **bidirectional transformation** between **RGB** and **YCbCr** and ensuring that our *evolutionary algorithm* can operate within this new space without altering the final CNN input format.

The goal is to introduce changes in **YCbCr**, but always evaluate the resulting image in **RGB** to ensure compatibility with pre-trained CNNs like ResNet-50.

4.1.1 From RGB to YCbCr and Back

The focus of this chapter is the **YCbCr color space**, widely used in digital image compression standards such as *JPEG*. Its structure separates **luminance (Y)** from **chrominance (Cb and Cr)** components, reflecting the way humans perceive images: we are more sensitive to brightness changes than to color differences.

This makes **YCbCr** especially interesting for adversarial image generation, as it suggests that perturbations introduced in certain channels (particularly **Cb** or **Cr**) may be **less perceptible to the human eye** [8], while still affecting the prediction of the model.

YCbCr components:

- **Y – Luminance:** encodes the brightness of the pixel.
- **Cb – Blue-difference chrominance:** encodes how blue the pixel is relative to luminance.
- **Cr – Red-difference chrominance:** encodes how red the pixel is relative to luminance.

Humans are much more sensitive to changes in the **Y** channel than in **Cb** or **Cr**, which makes it a promising strategy to generate perturbations in the chrominance channels to maximize *stealthiness* while preserving *attack efficiency*.

4.1.2 Algorithm Adaptation

To apply our evolutionary adversarial algorithm in the **YCbCr** space, we first need to convert an **RGB tensor** into **YCbCr**, and vice versa. This transformation is performed via **matrix multiplication**, as each YCbCr component is a weighted combination of the RGB channels (see Figure 4.1).

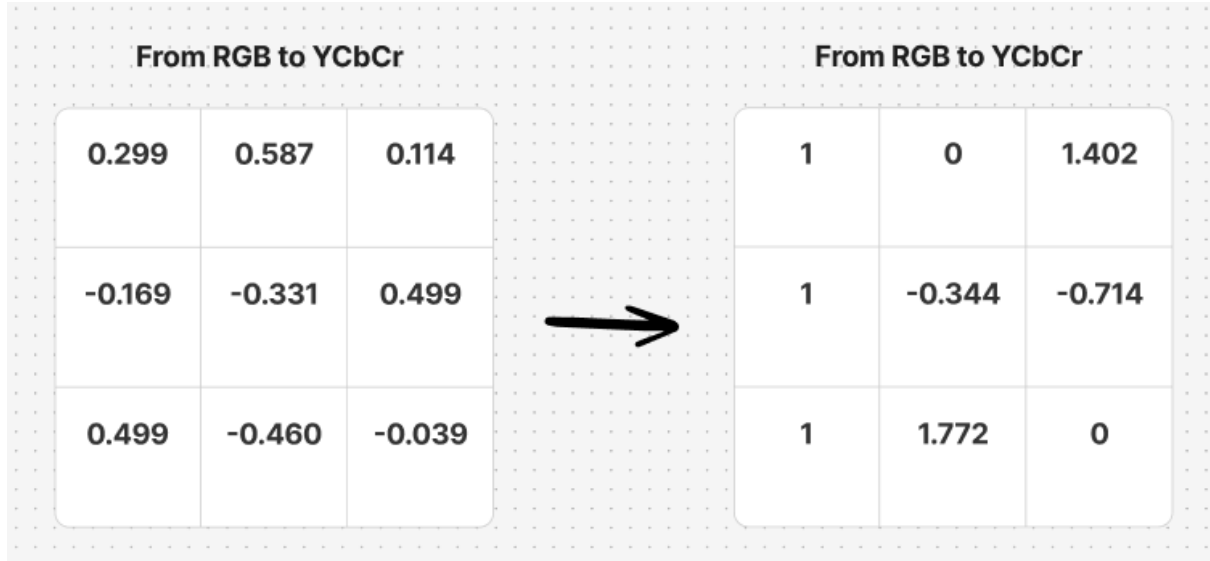


Figure 4.1: Transformation matrices for RGB to YCbCr and YCbCr to RGB color space conversions.

Now that we can convert images from RGB to YCbCr and back, we adapt our pipeline as follows:

- Convert the input image from **RGB** to **YCbCr**;
- Pass the **YCbCr tensor** to the evolutionary algorithm described in Chapter 2;
- Before each prediction, convert the tensor back to **RGB**, as the CNN only accepts RGB-encoded images.

It is also important to note that before clamping, we convert the batch of images back to the RGB color space. This step ensures that the added noise remains within controlled and visually plausible bounds.

The reason for this conversion lies in the nature of the YCbCr color space itself. Each of its three components (luminance Y , blue-difference chroma Cb , and red-difference chroma Cr) is mathematically derived from a combination of the original RGB channels. As a result, directly clamping values in the YCbCr space may lead to unexpected visual artifacts once the image is reconverted to RGB. In particular, certain color components can be overly exaggerated or distorted, even if each YCbCr value is within a valid range.

Returning to the RGB space prior to clamping ensures that the final color values remain within acceptable visual limits, preserving the perceptual realism of the adversarial image.

4.2 Experimental Objective and Setup

Having adapted our adversarial generation algorithm to operate within the **YCbCr color space**, the next step is to rigorously evaluate whether this transformation yields practical benefits over the traditional **RGB-based approach**.

This section presents a series of carefully designed experiments aimed at comparing the two methods along several critical dimensions: **convergence speed**, **effectiveness in misleading the CNN**, and **perceptual quality** of the resulting adversarial images.

We begin by measuring convergence behavior under limited computational constraints, followed by a more exhaustive analysis over **1000 epochs** to establish long-term efficiency. Finally, we assess the **visual imperceptibility** of adversarial perturbations applied exclusively to specific color channels.

These experiments serve not only to validate the use of YCbCr for adversarial attacks, but also to identify promising strategies for generating high-quality, stealthy adversarial examples.

4.2.1 Objective of the Experiment

The goal of this experiment is to evaluate whether adversarial images generated in the **YCbCr** space perform better than those generated in **RGB**. Performance can be measured in terms of:

- **Faster convergence**: fewer epochs to reach a successful attack;
- **Higher fooling rate**: better misclassification success;
- **Improved visual quality**: less perceptible perturbations, particularly when targeting the **Cb** and **Cr** channels.

If the results are promising, we will further investigate how to generate adversarial perturbations exclusively in the **least perceptible YCbCr channels** to produce adversarial examples that are both effective and visually indistinguishable from the original image.

4.2.2 Execution Time Benchmarking

All experiments in this chapter were conducted using the same hardware and software configuration detailed in subsection 2.3.1. The CNN architecture, hyperparameters, and image inputs remained identical across experiments to ensure fair comparison and reproducibility.

However, since the internal logic and computational complexity of each adversarial generation algorithm vary from chapter to chapter, the runtime per epoch may differ significantly.

To provide a concrete reference for performance evaluation, we report below the average execution time (in seconds) required to complete 100, 1,000, and 10,000 epochs for the algorithm implemented in this chapter:

- **100 epochs:** 67 seconds
- **1,000 epochs:** 777 seconds
- **10,000 epochs:** 7163 seconds

These measurements were obtained by running the full training loop on a fixed input batch of 40 images targeting class 285 (Egyptian Cat), using the standard hyperparameters. They serve as a useful metric for assessing the scalability and efficiency of the method in practical use cases.

4.2.3 First Experience

We will begin by examining the effectiveness of directly generating adversarial noise in images encoded in the **YCbCr color space**. To assess this effectiveness while mitigating the computational load of the experiment, we will first focus on comparing the convergence behavior of both algorithms, one operating in **RGB** and the other in **YCbCr**—towards **label 285**. This allows us to establish a point of comparison with the results from previous chapters.

To ensure that the experiment remains feasible in terms of runtime, we limit the number of epochs to **100**. The base parameters used will remain the same as those employed in prior experiments. We will repeat the test **10 times in 10 different images**, and compute the **average probability assigned to the label 285**. This should allow us to draw an initial conclusion regarding the potential benefits of using the YCbCr color space for adversarial noise generation (see Table 4.1).

Prediction RGB	Prediction YCbCr
0.00130	0.000634
0.00210	0.00472
0.000775	0.000991
0.000872	0.000895
9.83×10^{-5}	0.00739
8.21×10^{-5}	0.000932
4.35×10^{-5}	0.000684
0.00152	0.00186
1.60×10^{-5}	0.000192
6.66×10^{-5}	0.00252

Table 4.1: Progression of target label predictions during the first 100 epochs: comparison between the baseline algorithm and the YCbCr-based variant.

As we can observe, the **YCbCr version** of the algorithm outperforms the RGB version in **90% of cases**, indicating promising potential. For now, we can conclude that the YCbCr-based algorithm **converges more rapidly**. This is likely due to the fact that YCbCr is a **linear transformation** of RGB, and thus each modification in the Y, Cb, or Cr channels results in changes across all three RGB channels. Consequently, each mutation in YCbCr space affects a **broader range of pixel values** in RGB space, effectively enlarging the exploration scope, even when only 1% of the image is being altered.

This observation implies that after 100 epochs, the RGB-converted image, originally modified in the YCbCr space, is likely to have undergone **more widespread pixel-level changes** than its counterpart in the pure RGB-based algorithm. This supports our conclusion from Chapter 2 and the second experiment therein, which suggested that **modifying a higher percentage of the image often yields better results**.

4.2.4 Second Experience

To ensure that the **YCbCr version** of the algorithm is indeed more effective than the RGB version, it is essential to repeat the same experiment using **1000 epochs**. After such an extensive number of iterations, we can reasonably assume that nearly **100% of the pixels** have been modified in both variants of the algorithm. This will allow us to draw a more reliable and conclusive comparison regarding the true efficiency of the YCbCr-based algorithm (see Table 4.2).

Prediction RGB	Prediction YCbCr
0.133	0.674
0.130	0.957
0.0204	0.0328
0.0122	0.0325
0.00144	0.0196
0.00176	0.0406
0.00144	0.0252
0.0382	0.0836
0.00370	0.178
0.0137	0.118

Table 4.2: Prediction trajectories for the target label over 1000 epochs: baseline algorithm versus YCbCr-based algorithm.

As we can observe once again, the YCbCr-based algorithm proves to be more effective than the standard RGB version in **10 out of 10 cases**. This strongly confirms its superior efficiency in terms of convergence speed toward the target label. On average, the YCbCr version of the algorithm appears to be approximately **10.9 times more effective** than its RGB counterpart, which is a remarkable improvement.

However, it is important to note that this performance difference can be partly explained by the distinct behavior of noise propagation in the two color spaces. In both versions of the algorithm, the same **reach** parameter was used to sample additive noise uniformly from the range $[-reach, reach]$. This noise was then clamped to ensure it remained within bounds.

In the RGB-based version, this process directly perturbs the pixel values, resulting in strictly limited random fluctuations to the defined range.

By contrast, in the YCbCr-based version, the noise was added in the YCbCr domain before being converted back to RGB for clamping. Due to the linear transformation between YCbCr and RGB, even moderate noise applied in the luminance or chrominance channels (particularly *Cb* and *Cr*) can produce amplified changes in multiple RGB channels. As a result, the RGB values derived from the perturbed YCbCr representation are more likely to reach the clamping boundaries, effectively pushing the perturbed image to the extremes of the allowed range.

This difference implies that, although both versions technically apply noise of the same magnitude in their respective native spaces, the **YCbCr-based approach results in a larger effective perturbation in the RGB space**, thus accelerating the convergence

process.

However, the magnitude of the improvement, being approximately **10.9 times more effective**, cannot be solely attributed to this difference in noise propagation. In contrast, it highlights a key insight: *the diversity and randomness of the pixel variations alone are not what facilitates the discovery of adversarial examples*. What truly matters is the **total amount of effective perturbation**, provided that it remains imperceptible to the human eye. This is further evidenced by the fact that the use of a fixed **reach** value of 0.03 yields significantly better results in the YCbCr domain, without increasing perceptibility.

In other words, **maximizing the total amplitude of the perturbation, while remaining within perceptual constraints, is a much more effective strategy than relying on raw randomness**, reinforcing the idea that adversarial susceptibility is fundamentally driven by the accumulated shift in the characteristics relevant to the decision, as perceived by the convolutional neural network.

It is also important to emphasize that achieving a truly equivalent clamping strategy between the two color spaces is **extremely difficult, if not impossible in practice**. This is because the effect of a fixed perturbation in the YCbCr domain depends not only on the magnitude of the noise but also on the local structure of each pixel in the RGB space. Due to the non-orthogonal nature of the YCbCr-to-RGB transformation, the same perturbation in YCbCr can lead to widely different effects across pixels.

To illustrate this, consider two simple examples. Adding +0.1 to all three RGB channels of a neutral gray pixel $[0.5, 0.5, 0.5]$ increases only the luminance component Y , leaving the chrominance untouched. However, applying the same +0.1 shift to a blue pixel $[0, 0, 0.5]$ significantly increases not only the luminance Y but also the Cb component, since blue is strongly represented in chrominance. In other words, the perceptual and numerical consequences of a given RGB perturbation are highly dependent on the original pixel color.

Consequently, implementing a dynamic, pixel-specific clamping mechanism that ensures a perceptually and numerically equivalent perturbation across color spaces is computationally impractical. For this reason, a one-to-one comparison between RGB and YCbCr perturbations, under a shared clamping constraint, is fundamentally beyond our reach.

The initial motivation behind this experiment was based on the perceptual insight that perturbing only the chrominance channels (that is, Cb and Cr) might allow stronger noise injection without being visually noticeable. The underlying assumption was that these channels, being less critical to human perception than luminance, could absorb

more variation without degrading the visual quality of the image. However, a deeper analysis reveals a conceptual nuance: Modifying chrominance does not add noise in the traditional sense; it simply alters the balance between the red, green, and blue channels in a way that preserves the overall brightness of the pixel.

For example, consider a neutral gray pixel with RGB values $[0.5, 0.5, 0.5]$. Changing its chrominance by increasing Cr while compensating with a corresponding shift in G and B can transform it into a warmer tone (slightly red-shifted), yet the average intensity across channels, and therefore the perceived luminance, remains unchanged. In essence, chrominance manipulations rotate the color tone without increasing the total energy of the perturbation.

The preceding results suggest that what truly influences the CNN prediction is not the direction of the color change, but rather the **total magnitude of the change applied to the image**, regardless of how perceptible it is to the human eye. This supports the idea that adversarial vulnerability is more sensitive to the absolute accumulation of pixel-level variations than to perceptual realism.

4.2.5 Third Experience

To clarify the effect of manipulating chrominance versus luminance, we designed a dedicated experiment. Rather than converting the image batch to the YCbCr color space to modify the chrominance components, we chose to remain in the RGB space. However, we introduced an important constraint: the **sum of the three RGB channels for each pixel must remain constant**. This ensures that the perceived luminance is preserved, allowing us to isolate the impact of color shifts (i.e., chromatic variations) without altering brightness.

Moreover, to make this new experiment directly comparable with the previous one, conducted in RGB with a fixed **reach** value of 0.03, we maintained an equivalent constraint on the magnitude of the perturbation. Recall that in the RGB setup, each channel could be modified within the interval $[-0.03, 0.03]$, which implies that the cumulative modification of a pixel could reach values between -0.09 and 0.09 (i.e., $3 \times \text{reach}$).

In our current experiment, we therefore ensure that for every modified pixel, the total sum of the changes across its R, G, and B channels falls within this same range of $[-0.09, 0.09]$, and the final sum of the RGB values remains equal to the original one. In this way, we isolate the chromatic shift while maintaining both luminance and perturbation equivalence, making the comparison both fair and interpretable.

To implement this chrominance preservation strategy, we randomly selected a subset

of pixels, typically 1% of the image, and introduced color perturbations that satisfy two critical conditions.

- the sum of the three RGB channels for each pixel remains unchanged (i.e., $\Delta R + \Delta G + \Delta B = 0$), and
- each individual channel stays within a bounded range around its original value (e.g., $R' \in [R - \delta, R + \delta]$, with $\delta = \text{chrome_reach}/2$).

For each pixel, a noise vector $\boldsymbol{\delta} \in R^3$ is sampled such that:

$$\sum_{i=1}^3 \delta_i = 0 \quad \text{and} \quad \|\boldsymbol{\delta}\|_1 = \text{chrome_reach}$$

This is achieved by sampling a raw vector $\mathbf{u} \sim \mathcal{U}(-1, 1)^3$, then centering it to enforce zero mean:

$$\mathbf{u} \leftarrow \mathbf{u} - \text{mean}(\mathbf{u})$$

and finally normalizing it to enforce a fixed ℓ_1 norm:

$$\boldsymbol{\delta} = \frac{\mathbf{u}}{\|\mathbf{u}\|_1} \cdot \text{chrome_reach}$$

The resulting vector $\boldsymbol{\delta}$ is then added to the original RGB value. If any of the new channel values exceed the image-bound constraints $[0, 1]$ or the perturbation limits allowed per channel, the vector is discarded and resampled. This rejection sampling guarantees that all perturbed pixels respect both the chrominance and perceptual validity constraints.

Naturally, `chrome_reach`, which defines the total absolute noise added per pixel, will be set to 0.09. This value corresponds to the maximum cumulative noise that can be introduced when using a `reach` of 0.03, as each of the three RGB channels can receive up to 0.03 of perturbation.

While this implementation is conceptually precise and well-aligned with the question under investigation, it comes with a significant computational drawback. The constraints imposed on each pixel’s perturbation vector, namely, preserving luminance while maintaining a fixed perturbation magnitude, are so restrictive that satisfying them often requires a large number of rejection-sampling iterations.

As a result, runtime increases dramatically. For example, when running the baseline RGB algorithm with the shared hyperparameters and 100 epochs, the total runtime is

approximately 20 seconds. In contrast, applying the chrominance-constrained strategy increases the run time to nearly 1,000 seconds, a 50 fold increase.

Due to this substantial computational cost, we were forced to design a reduced experimental setup. In this section, we therefore report the results obtained after 100 epochs over only 5 input images. All other hyperparameters remain identical to those described in subsection 2.3.1. To ensure that the reported results are representative and not influenced by outliers, the experiment was repeated 10 times, and the average performance was computed across runs (see Table 4.3).

Prediction RGB	Prediction Chrominance
0.00115	9.86×10^{-4}
0.00219	0.00317
6.34×10^{-4}	4.13×10^{-4}
8.66×10^{-4}	5.93×10^{-4}
1.18×10^{-5}	3.03×10^{-6}

Table 4.3: Prediction trajectories for the target label over 100 epochs: baseline algorithm versus chrominance algorithm.

As we can observe, the results obtained with this method are, on average, approximately **1.75 times less effective** than those of the baseline algorithm. This outcome is not surprising, considering that luminance remains unaltered in our approach. Nonetheless, the results may still be regarded as encouraging, as they demonstrate that even without modifying the luminance, the performance gap remains relatively moderate.

It is important to recall the primary motivation behind this experiment. The goal was not to surpass the baseline algorithm in raw performance, but rather to develop a more perceptually subtle method. Although heavier in computational cost and slightly less effective, the proposed algorithm perturbs only the chrominance channels, making the resulting adversarial modifications significantly less perceptible to the human eye.

While perceptual sensitivity is inherently subjective and challenging to quantify precisely, a visual comparison can still provide valuable insights. Below, we present two adversarial images: one generated by modifying both luminance and chrominance, and the other by altering only the chrominance. Both images were crafted using equivalent noise budgets, ensuring that the total perturbation per pixel did not exceed 0.21, setting the `reach` parameter to 0.07 for standard RGB noise and setting `chrome_reach` to 0.21 in the version with chrominance only. This guarantees a fair side-by-side evaluation under consistent perturbation constraints (see Figure 4.2 and 4.3).



Figure 4.2: Adversarial image generated by modifying both chrominance and luminance components.



Figure 4.3: Adversarial image generated by modifying only the chrominance components (Cb and Cr), with luminance preserved.

As we can observe and although this judgment remains inherently subjective, the image in which only the chrominance has been modified appears to exhibit a slightly more visible noise pattern, characterized by a greater diversity of colors that, despite being subtle, remain perceptible to the human eye.

In conclusion, despite the considerable effort devoted to this experiment, and even

if we set aside the observed drawbacks in terms of both performance and runtime, the visual outcome of this chrominance-only algorithm does not appear to offer any significant advantage. Its perceptual impact is not significantly different, if not slightly inferior, from that of the baseline method. For this reason, we conclude that restricting perturbations exclusively to chrominance channels does not bring tangible benefit in the context of adversarial image generation using evolutionary algorithms.

4.2.6 Conclusion

Through a series of targeted experiments, we have systematically evaluated the relevance of generating adversarial images in the **YCbCr color space** as opposed to the traditional **RGB** space. Our results reveal that modifying images in YCbCr leads to significantly faster convergence and improved attack success, even when identical noise amplitudes are used. This performance gain is attributed not only to the structure of the YCbCr transformation, which causes each perturbation to influence multiple RGB channels simultaneously, but also to the more efficient use of the allowable perturbation space.

However, when we constrained the attack to alter only the chrominance channels (Cb and Cr) while preserving luminance, the results were more nuanced. Despite the appeal of this strategy from a perceptual standpoint, the algorithm proved computationally expensive and somewhat less effective than the baseline RGB method. Moreover, the visual difference between the images generated with and without luminance modification was subtle and did not justify the significant increase in runtime.

In summary, while the YCbCr color space offers a promising alternative to crafting stronger adversarial perturbations, focusing solely on chrominance does not appear to offer a meaningful advantage, either in terms of stealth or performance, within the evolutionary framework explored in this study.

Chapter 5

Deterministic Pixel-Wise Adversarial Image Generation

In contrast to the stochastic or heuristic-based strategies explored in the preceding chapters, this final approach to adversarial image generation is entirely **deterministic**. Instead of relying on randomness or guided evolution, we directly interrogate the model to uncover which pixels, when altered in a binary fashion, most effectively steer its prediction toward a target label.

This chapter introduces an **extensive binary probing strategy** in which the individual contribution of each pixel is tested through controlled modifications. The method capitalizes on the knowledge that the directionality of a pixel (toward 0 or 1) can be deterministically evaluated by comparing the CNN’s output on *lit* versus *dark* variants of that pixel.

Although computationally intensive, this strategy benefits from **reproducibility**, **interpretability**, and **fine-grained control**. It brings us closer to a brute-force adversarial search, yet in a surprisingly tractable and scalable manner thanks to effective implementation optimizations.

5.1 Exhaustive Binary Pixel Probing Strategy

The experiments that follow explore the feasibility, effectiveness, and visual subtlety of adversarial images generated using this probing technique. We investigate whether this deterministic method can match or even surpass the performance of prior strategies, particularly in terms of **attack reliability** and **computational efficiency** under constrained resources.

This section provides a detailed description of the algorithm, including its pixel-wise evaluation logic, performance trade-offs, and an empirical comparison to evolutionary methods presented earlier in this report.

5.1.1 Introduction

In this final Chapter 5, we present our last experiment aimed at generating adversarial images. This method is vastly different from the others as it does not rely on evolutionary algorithms or even randomness. Instead, it stems from a simple idea: rather than guessing the best adversarial image through numerous iterations and random noise, we attempt to directly find it by testing each individual pixel.

5.1.2 Algorithm Description

Our algorithm is relatively simple. For each channel of every pixel, we generate two alternative versions of the image: one where that specific channel of that pixel is fully lit (i.e., its value is set to 1), and another where it is completely turned off (i.e., its value is set to 0). We then send these two images, as a batch, through the CNN to observe which one achieves a higher prediction probability for a specified label.

Ultimately, this process is similar to recreating gradient directions, similar to those used internally by CNNs. By doing this for each pixel and each channel, it is as if we are asking the model: *'In which direction should I modify this pixel to increase the probability of label 285?'*

5.1.3 Advantages, Constraints, and Precision

Naturally, this approach comes with both pros and cons. The main advantage is that the results are fixed and reproducible. However, the major drawback is its computational cost. Initially, we thought the method would be too heavy to implement in practice, but I was positively surprised. I spent a lot of time optimizing and refining the approach, using PyTorch for efficiency.

To give you a sense of scale: we must send $224 \times 224 \times 3 \times 2 = 301,056$ images through the CNN. On the other hand, in the worst-case scenario of the evolutionary algorithms described in previous chapters, we process $10,000 \text{ epochs} \times 80 \text{ images}$ (since we check the batch twice per epoch), totaling 800,000 images. Thus, if well-optimized, our method could be more efficient than the ones previously introduced.

For additional detail, we randomly sample a percentage of pixel coordinates. For each coordinate, we generate the two aforementioned image versions (lit and dark) and send them through the CNN. We then retain the higher probability between the two. After collecting the best results for each coordinate, we select a percentage of them and apply perturbations in the correct direction (increased or decreased) using the predefined **reach** value.

This process may sound complex, so here is an example: suppose we test the coordinate (1, 100, 100) and find that the “dark” version of the image (with this pixel turned off) increases the probability of label 200. As a result, we reduce the pixel’s value by **reach** to move it toward 0, thereby potentially nudging the CNN’s prediction toward the target class.

Note that we use two distinct percentage parameters in this algorithm: one to define the proportion of coordinates to test and another to determine the best performing subset that will actually be modified. This two-step filtering is necessary because we feared that the algorithm might be too heavy to run on 100% of the image coordinates.

To illustrate the two-stage selection process more clearly: in the first step, we randomly sample 10% of the total coordinates from the image. In the second step, we compute the effect of each of these sampled pixels on the CNN’s prediction and proceed to modify only the top 50% most influential ones. This approach allows us to focus the perturbation on the regions of the image that contribute most strongly to the model’s decision.

5.2 Experimental Objective and Setup

In this section, we outline the experimental setup and objectives for evaluating the deterministic pixel-wise adversarial algorithm introduced earlier. Through a series of progressively more complex experiments, we aim to assess the efficiency, reliability, and adaptability of this method under varying conditions and parameter configurations.

By systematically exploring its behavior, we hope to identify optimal settings that balance computational cost with adversarial success rate, ultimately demonstrating whether this deterministic approach offers a viable alternative to previously discussed algorithms.

5.2.1 Objective of the Experiment

Before diving into the experimental results, it is important to clearly define our goal. The purpose of this experiment is to find an alternative and reproducible method to evolutionary algorithms for generating adversarial images. We aim to either achieve faster results or produce higher quality adversarial samples.

5.2.2 Execution Time Benchmarking

As with all other experiments in this study, the benchmarks reported below were performed on the same hardware and software setup described in subsection 2.3.1. The CNN architecture, input images, and hyperparameters remained unchanged.

However, unlike previous algorithms that iterate over a predefined number of epochs, the computational cost of the *best_pixels* algorithm is almost entirely determined by the number of forward passes through the CNN. This is because each coordinate of the pixel channel must be individually evaluated twice, once in its *lit* form and once in its *dark* variant, resulting in a significant computational load.

To assess performance, we report execution times for evaluating three different proportions of the image coordinates:

- **25% of the coordinates:** $0.25 \times (224 \times 224 \times 3 \times 2) = 75,264$ CNN forward passes
- **50% of the coordinates:** $0.5 \times (224 \times 224 \times 3 \times 2) = 150,528$ CNN forward passes
- **100% of the coordinates:** $1.0 \times (224 \times 224 \times 3 \times 2) = 301,056$ CNN forward passes

In all cases, we compute the worst-case scenario in which **100% of the evaluated coordinates are modified**. Although the act of modifying these coordinates has a negligible effect on runtime, this assumption ensures a consistent and conservative estimate of total processing cost.

The measured execution times for each case are as follows:

- **25% (75,264 passes):** 655 seconds
- **50% (150,528 passes):** 1332 seconds

- **100% (301,056 passes):** 2674 seconds

These timings provide a realistic estimate of the algorithm’s computational requirements under varying levels of precision, and serve as a benchmark for evaluating scalability across future extensions of the method.

5.2.3 First Experience

Our first experiment aims to explore the behavior of the algorithm in various combinations of parameters. In order to obtain a clear and precise initial overview, we will test all possible combinations of the parameters listed in the figure below (see Figure 5.1).

```
list_pourcentages = np.arange(0, 1.05, 0.05)
list_reach = np.arange(0, 0.11, 0.01)
```

Figure 5.1: List of tested values for the percentage and reach parameters during the initial experiment.

A single fixed image will be used throughout the experiment and each configuration will be tested 10 times. We will then compute the average of these 10 runs to ensure that the results are stable and provide a reliable preliminary assessment of the algorithm’s behavior (see Figure 5.2).

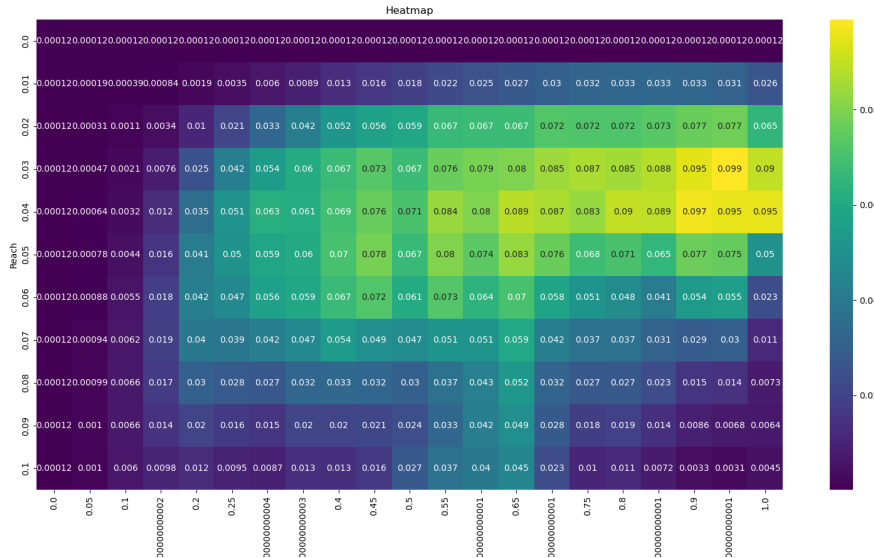


Figure 5.2: Heatmap of ResNet-50 predictions for class 285 across combinations of percentage and reach parameters.

The results obtained are both surprising and interesting. Initially, one might expect that increasing the intensity of the noise represented by the parameter `reach` would bias the CNN more effectively, leading to stronger convergence towards the target label. However, this did not turn out to be the case. The algorithm actually achieved better results with a relatively low `reach` value of 0.03.

Even more impressively, it appears that modifying 100% of the image is not necessarily the best strategy. As the results suggest, modifying 90% to 95% of the image can be as effective, or even more so. These findings are counterintuitive and highlight the complexity of CNN behavior.

Furthermore, a clear trend emerges, with a performance peak around the coordinates `reach = 0.03` and `min_percentage = 0.95`. This suggests that the behavior of the algorithm is not random, but instead follows a predictable pattern.

In terms of actual performance, the best result achieved was a probability of 0.1 for the target label 285. Although this may seem low compared to the algorithms used in previous chapters, it is important to note that this result was obtained in a single iteration, making it remarkably efficient.

5.2.4 Second Experience

Our next experiment takes the idea even further by mimicking the behavior of the gradients used in CNN training. Specifically, we iteratively compute the optimal modification to apply for 90% of the most influential pixels in the image, repeating this process five consecutive times. Importantly, instead of restarting from the original image at each iteration, we continue to modify the previously altered image.

We maintain a fixed step size of `reach = 0.01` throughout the process. This setup allows us to investigate whether such a small modification step can gradually guide the image toward a local minimum, and whether the behavior of our algorithm truly resembles the backpropagation mechanism inherent in CNNs (see Table 5.1).

Iteration	Prediction
0	0.0326
1	0.660
2	0.989
3	0.999
4	1.00

Table 5.1: Predicted confidence for class 285 when modifying the top 90% most influential pixels among 100% of the image, using a step size of 0.01.

As shown in the previous experiment, the results are quite impressive: after only three iterations, achieving a prediction confidence of 98% for the target label is a strong indicator of the algorithm’s effectiveness. However, a significant limitation arises, each iteration requires passing 301,056 images through the CNN. Consequently, after three iterations, the total number of forward passes exceeds 900,000, which makes the algorithm computationally heavier than the evolutionary algorithm, which typically peaks at around 800,000 passes.

5.2.5 Third Experience

To address this limitation, we designed a follow-up experiment aimed at reducing the computational burden by half. Instead of testing 100% of the image pixels, we will randomly sample and evaluate only 50% of the total pixels. As before, we will select and perturb the top 90% most influential pixels within the sampled set.

In summary, in the previous experiment, we tested all pixels to select and modify 90% of the most influential ones. In the present experiment, we will test only 50% of the pixels and still modify the 90% most impactful among them. This setup allows us to observe whether the results remain similar or if the final adversarial confidence drops proportionally. For consistency and fair comparison, the step size (**reach**) will remain unchanged at 0.01 (see Table 5.2).

Iteration	Prediction
0	0.0066
1	0.157
2	0.716
3	0.977
4	0.998

Table 5.2: Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 50% of the image, with a step size of 0.01.

As we can observe, the results are highly promising. Each iteration corresponds to approximately 150,000 forward passes through the CNN. This means that we successfully generated an adversarial image with a prediction confidence of around 98% for the target label in only four iterations, totaling roughly 600,000 passes. In contrast, the previous experiment required over 900,000 passes to reach a similar result.

This raises several important uncertainties. In the second experiment, the majority of pixels underwent three consecutive mutations of 0.01 (step size), given that we reached a high prediction score after three iterations. However, we achieved a comparable result after four iterations in the third experiment, despite modifying only 45% of the pixels per iteration (90% of 50%).

This implies that the total amount of noise applied in the second experiment is significantly lower than in the first. Specifically, the cumulative noise for the first experiment is approximately:

$$3 \times 0.01 \times 0.9 \times 224 \times 224 \times 3 = 4,064$$

while in the second experiment, it is:

$$4 \times 0.01 \times 0.45 \times 224 \times 224 \times 3 = 2,710$$

However, it is important to note that in the worst case scenario, a small fraction of the image, ranging from 0 to 45%—may still contain pixels with a cumulative noise as high as 0.04. This observation underlines the non-uniform distribution of perturbations across the image and suggests that some regions may have concentrated changes despite an overall reduced noise budget.

It is also important to note that in the previous experiment, the set of pixels representing 45% of the image was randomly selected at each iteration. With such promising

results, a natural question arises: What happens if the same set of pixels is kept constant throughout all iterations, rather than being reselected randomly each time?

In this configuration, the noise will accumulate on the same pixels throughout iterations, which could increase the chances of generating a high-quality adversarial image in fewer than four iterations, therefore requiring fewer than 600,000 CNN forward passes. However, since the pixel set remains fixed, there is also a risk of selecting suboptimal pixels. To address this uncertainty, we repeated the experiment five times and computed the average confidence for label 285 at each iteration.

Moreover, as there is little value in extending the experiment beyond the third iteration in this scenario, we limited the analysis to three iterations.

The results of this third experiment are shown below (see Table 5.3).

Iteration	Prediction
0	0.00636
1	0.148
2	0.723

Table 5.3: Predicted confidence for class 285 when modifying the top 90% most influential pixels among a fixed 50% subset of the image, with a step size of 0.01.

As we can observe, the results obtained by applying noise to a fixed region are very similar to those obtained when the region is randomly selected at each iteration. These results are quite surprising and once again suggest that what truly matters is the total amount of modification present in the image, as long as these modifications are properly oriented toward the local minimum.

That being said, this suggests that with this method of adversarial image generation, it would be feasible to generate effective adversarial examples even when modifying only a portion of the image while leaving the rest untouched. This property could prove useful in future applications where the preservation of certain regions of the image is necessary.

5.2.6 Fourth Experience

At this stage, two main challenges remain. First, our most efficient adversarial generation still requires approximately 600,000 CNN forward passes, which remains computationally expensive. Second, even if we further reduce the number of processed pixels per iteration to only 25% of the total image and continue to modify 90% of them, it may take more than 4 iterations to achieve competitive results. This means that the same pixels may

accumulate noise over several iterations, possibly leading to pixel saturation and visible distortions, especially with a step size of 0.01.

To address these concerns and better understand the behavior of the algorithm, we propose two new experiments:

1. In the first experiment, we reduce the number of pixels evaluated per iteration to 25% of the image. From this subset, we continue to modify 90% of the most influential pixels, using a step size of 0.01. The goal is to evaluate whether comparable results can be achieved with fewer CNN evaluations (see Table 5.4).
2. The second experiment mirrors the first, but with a halved step size of 0.005. This aims to mitigate the accumulation of excessive noise on the same pixels and observe whether smaller changes, accumulated over time, can still effectively increase the prediction probability for the target label (see Table 5.5).

Each of these experiments is conducted over 7 iterations, totaling approximately 525,000 CNN passes. Our objective is to determine whether we can surpass the previously established performance threshold of 600,000 evaluations with improved efficiency or better quality adversarial examples.

Iteration	Prediction
0	0.0015
1	0.0124
2	0.0706
3	0.296
4	0.648
5	0.882
6	0.968

Table 5.4: Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 25% of the image, using a step size of 0.01.

Iteration	Prediction
0	0.0005
1	0.0017
2	0.005
3	0.0135
4	0.0334
5	0.0785
6	0.174

Table 5.5: Predicted confidence for class 285 when modifying the top 90% most influential pixels among a randomly selected 25% of the image, using a step size of 0.005.

We begin with the first experiment, in which we successfully generated a high-quality adversarial image after only 525,000 CNN forward passes, corresponding to the seventh iteration. However, this also implies that the maximum noise accumulated on certain pixels could reach up to 0.07 (i.e., 7×0.01). To address this, we conducted a second experiment with a halved step size (reach = 0.005), aiming to mitigate the overall noise accumulation.

Unfortunately, the results of this second experiment were not as promising, as it failed to produce an effective adversarial image. This suggests that simply reducing the step size is not a viable solution.

In summary, we observed that reducing the number of evaluated pixels allows for fewer total forward passes through the CNN, thus improving the algorithm’s efficiency. However, the fixed step size (reach = 0.01) tends to accumulate over multiple iterations, especially when more iterations are required due to smaller pixel subsets. While reducing the step size could theoretically limit this accumulation, it also significantly impairs the algorithm’s performance. Therefore, a trade-off must be carefully considered between computational cost and the effectiveness of the generated adversarial image.

Therefore, it appears that evaluating 50% of the total image and subsequently modifying 90% of the evaluated pixels with a step size of 0.01 constitutes a viable parameter configuration. Although not the most optimal setting, it consistently yields near-optimal results. Given the computational burden and time-consuming nature of these experiments, this configuration offers a practical compromise between the subtlety of the resulting adversarial image and the overall performance of the algorithm. We will thus adopt this parameter set for our next experiment.

5.2.7 Fifth Experience

Our next experiment is relatively straightforward. Until now, we have limited ourselves to the same image and a single target label. We will now apply the algorithm, using the parameter configuration established previously, to generate 5 adversarial images corresponding to 5 randomly selected labels among the 1000 classes in ResNet-50. This process will be repeated for 5 different base images. This setup will allow us to assess whether the *best_pixels* algorithm is truly versatile and capable of generating effective adversarial examples across a wide range of target classes and input samples.

To present the results clearly, we will report the **average probability** and the **average number of iterations** required to generate an adversarial image for each target label. Additionally, we will include the **worst case scenario**, as it can provide valuable insights into the potential limitations of the algorithm (see Table 5.6).

Label	Iteration	Prediction
56	5	0.986
298	5	0.969
471	4	0.990
613	5	0.999
820	6	0.980

Table 5.6: Average number of epochs required to generate adversarial examples reaching over 90% confidence for various target labels, by modifying the top 90% most influential pixels within a randomly selected 50% subset of the image at each iteration, using a step size of 0.01.

The worst-case scenario required 6 epochs, which is relatively high. However, the average remained around 5 iterations, which corresponds to approximately 750,000 forward passes through the CNN. This is still below the 800,000 passes observed in the worst-case scenario of the evolutionary algorithm.

Moreover, the results are very promising: the average confidence score for the target label was consistently above 90% across all tested labels. Most importantly, an adversarial image was successfully generated for every randomly selected label, despite being significantly different from the original label of the input image.

These findings suggest that the *best_pixels* algorithm is both highly effective and versatile, capable of generating adversarial images regardless of the target label or base image.

5.2.8 Conclusion

Through this comprehensive series of experiments, we evaluated the performance, limitations, and potential of our deterministic pixel-wise adversarial generation algorithm, referred to as *best_pixels*. Several key findings emerged from this investigation.

First, we observed that even a single iteration of the algorithm, when appropriately configured, can produce nontrivial adversarial perturbations. Notably, perturbing only 90% of the most influential pixels yielded a significant increase in prediction confidence toward the target class, especially when combined with a modest noise amplitude ($\text{reach} = 0.03$). This efficiency highlights the algorithm’s potential for rapid, reproducible adversarial generation.

Second, by progressively modifying the image through iterations while using influence-based pixel selection, we demonstrated the algorithm’s ability to reliably produce high-confidence adversarial examples. Importantly, this was achieved even under strict constraints, such as limiting the modification to only 50% or 25% of the image pixels at each iteration. These findings suggest that our algorithm can maintain performance while significantly reducing the computational footprint.

Third, we explored the impact of noise distribution—both in terms of spatial locality and amplitude accumulation. Our experiments revealed that while fixed pixel sets can accelerate convergence by accumulating noise on a small region, randomly resampled pixel subsets often offer comparable performance while ensuring a more uniform perturbation distribution. We also showed that reducing the noise step size too aggressively hinders effectiveness, underscoring the delicate balance between subtlety and strength in adversarial design.

Finally, in our most general test, we applied the algorithm across a diverse set of labels and base images. The *best_pixels* method consistently produced adversarial images with over 90% target confidence, regardless of the semantic distance between the source and target classes. This robustness confirms its generalizability and effectiveness beyond narrowly defined conditions.

In summary, the *best_pixels* algorithm offers a promising deterministic alternative to evolutionary strategies. It is highly configurable, reproducible, and capable of generating powerful adversarial examples with a reduced computational cost. Its sensitivity to parameter tuning, particularly the selection percentage and noise amplitude, makes it a flexible yet precise tool for adversarial image generation. These findings open new avenues for fast, interpretable, and structured adversarial research.

5.3 General Conclusion

This study presented a comparative and multi-faceted exploration of adversarial image generation techniques, with the goal of improving both the effectiveness and perceptual subtlety of such attacks. Four distinct strategies were investigated, each addressing a specific aspect of adversarial design, from spatial targeting to color-space manipulation, universality, and deterministic optimization.

The first set of experiments demonstrated that **leveraging edge and contour information** can meaningfully guide perturbation placement, especially under constrained conditions. Although the edge-based strategy did not always outperform random baselines in unconstrained scenarios, it proved significantly more effective when noise was limited, confirming the hypothesis that CNNs are more vulnerable in regions rich in structural features.

The second line of investigation focused on **universal adversarial masks**, which aim to decouple perturbations from individual images. While these masks were not successful in reliably steering predictions toward a target label, they disrupted top-1 predictions in 30% of cases. This partial success suggests that although the approach lacks robustness in its current form, it holds promise with further refinement.

The third strategy explored the use of the **YCbCr color space** as an alternative domain for perturbation injection. The results showed that perturbations applied in this space converged significantly faster and yielded higher fooling rates than their RGB counterparts, due to a more distributed impact across channels. However, when restricting changes exclusively to chrominance components, the computational cost rose sharply and perceptual gains were minimal, ultimately calling into question the value of chrominance-only strategies.

Finally, we introduced a **deterministic, pixel-wise perturbation algorithm**, which directly measures the contribution of each pixel to model predictions. The method proved to be efficient and highly generalizable. It consistently produced high-confidence adversarial images across diverse targets and input images, while requiring fewer forward passes than traditional evolutionary approaches. Its deterministic nature and adaptability position it as a viable and interpretable alternative to stochastic methods.

Taken together, these experiments reveal a complex landscape in which the success of adversarial strategies depends heavily on both technical configuration and perceptual trade-offs. By systematically isolating and testing structural, spectral, universal, and deterministic components of adversarial generation, this work provides not only a set of novel techniques but also a clearer understanding of how and where to apply them most

effectively. The insights gained here lay the groundwork for future research aimed at building adversarial methods that are more efficient, interpretable, and ultimately, more aligned with real-world application constraints.

This report was written in English and formatted in L^AT_EX. Some translations and formatting were assisted by OpenAI’s ChatGPT [6]. All scientific content remains the sole responsibility of the author.

Bibliography

- [1] Moustafa Alzantot, Yash Sharma, Supriyo Chakraborty, Huan Zhang, Cho-Jui Hsieh, and Mani Srivastava. Genattack: Practical black-box attacks with gradient-free optimization. *arXiv preprint arXiv:1805.11090*, 2018.
- [2] James D. Foley, Andries van Dam, Steven K. Feiner, and John F. Hughes. *Computer Graphics: Principles and Practice*. Addison-Wesley Professional, 2nd edition, 1995. Alpha blending and compositing techniques.
- [3] N. Kanopoulos, N. Vasanthavada, and R. L. Baker. Design of an image edge detection filter using the sobel operator. *IEEE Journal of Solid-State Circuits*, 23(2):358–367, 1988.
- [4] Microsoft Research. Microsoft cat vs dog dataset. <https://www.microsoft.com/en-us/download/details.aspx?id=54765>, 2012. Accessed: 2025-06-05.
- [5] Author Names. One evolutionary algorithm deceives humans and ten convolutional neural networks trained on imagenet at image recognition. *Applied Soft Computing*, 2023.
- [6] OpenAI. ChatGPT (Version 4). <https://chat.openai.com>, 2024. Accessed via <https://chat.openai.com>.
- [7] Orient Display. 1.5" ips display, 240x240, 255 nits, mcu/spi interface, 2021. Consulté le 5 juin 2025.
- [8] Charles Poynton. Video systems in television. *Wiley Encyclopedia of Electrical and Electronics Engineering*, 22:1–19, 1998. Explains that the human eye is less sensitive to chrominance (Cb, Cr) than luminance (Y).
- [9] Alberto Salmerón. Filtros de imágenes en julia, 2021. Consulté le 5 juin 2025.
- [10] Alberto Salmerón. Filtros de imágenes en julia, 2021. Consulté le 5 juin 2025.

- [11] Ali Osman Topal, Raluca Chitic, and Franck Leprévost. One evolutionary algorithm deceives humans and ten convolutional neural networks trained on imagenet at image recognition. *Applied Soft Computing*, 143:110397, 2023.
- [12] Yifan Wang, Yuchao Li, and Lei Zhang. Unlocking the black box of cnns: Visualising the decision-making process of convolutional neural networks. *Information Sciences*, 622:1–14, 2023.